Microsoft

(DP-800)

Developing AI-Enabled Database Solutions

Total: **105 Questions**

Link: https://examauthor.com/papers/microsoft/dp-800

Question: 1 **Exam Author**

HOTSPOT -
Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to meet the development requirements for the FeedbackJson column.
How should you complete the Transact-SQL query? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

```
SELECT
  f.FeedbackId,
  f.VehicleId,
```

[▼] ,

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

```
  EDIT_DISTANCE_SIMILARITY(
    JSON_VALUE(f.FeedbackJson, '$.text'),
    @KnownIssueDescription
  ) AS SimilarityScore
FROM
  dbo.CustomerFeedback f
WHERE
```

[▼]

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

```
ORDER BY
```

[▼]

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3

```
EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3

JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText

JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText

SimilarityScore
```

```
DESC;
```

Answer:

Answer Area

```
SELECT
  f.FeedbackId,
  f.VehicleId,
  [ ],
  EDIT_DISTANCE_SIMILARITY(
    JSON_VALUE(f.FeedbackJson, '$.text'),
    @KnownIssueDescription
  ) AS SimilarityScore
FROM
  dbo.CustomerFeedback f
WHERE
  [ ]
ORDER BY
  [ ]
DESC;
```

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

Explanation:

Box 1 (SELECT clause): JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText

Extracts the raw string from the FeedbackJson object to provide the source text needed for the EDIT_DISTANCE_SIMILARITY function immediately following it.

Box 2 (WHERE clause): EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3

Filters records to only include rows where the edit distance (Levenshtein distance) between the text field and the @Keyword is strictly less than 3.

Box 3 (ORDER BY clause): SimilarityScore

Reuses the column alias defined in the SELECT block (AS SimilarityScore) to sort the output in descending order (DESC).

Why the other answer are incorrect:

CONTAINS(FeedbackJson, @Keyword): Invalid output type. It returns a boolean truth value (1 or 0) for full-text searches instead of a text value.

EDIT_DISTANCE(...) < 3: Syntax error. This is a logical filter condition (predicate) and cannot be named directly as a string column expression.

JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription): Wrong data type. JSON_QUERY only extracts valid objects or arrays. It returns NULL for scalar text properties.

SimilarityScore: Reference error. You cannot select the alias itself before its computation expression is declared.

CONTAINS(FeedbackJson, @Keyword): Wrong path context. It checks the entire JSON string instead of isolating the necessary $.text path property.

EDIT_DISTANCE(..., '$.details.comment', ...): Property mismatch. This evaluates a completely different nested field (comment), ignoring the target $.text property specified by the query logic.

JSON_QUERY(...) / JSON_VALUE(...): Structural syntax error. These functions extract data values. A WHERE clause demands a boolean predicate (<, >, =).

SimilarityScore: Execution order failure. Standard SQL processing evaluates WHERE long before SELECT. The engine does not yet recognize the column alias.

CONTAINS(...) / EDIT_DISTANCE(...) < 3: Logic mismatch. These evaluate to true/false criteria. They do not provide a continuous numeric spectrum needed to rank similarity accurately.

JSON_QUERY(...) / JSON_VALUE(...): Sort goal failure. Ordering by the raw text characters sorts alphabetically instead of ranking items by how closely they match the keyword.

Question: 2 **Exam Author**

DRAG DROP -
You have an Azure SQL database that contains a table named dbo.Orders.
You have an application that calls a stored procedure named dbo.usp_CreateOrder to insert rows into dbo.Orders.
When an insert fails, the application receives inconsistent error details.
You need to implement error handling to ensure that any failures inside the procedure abort the transaction and return a consistent error to the caller.
How should you complete the stored procedure? To answer, drag the appropriate values to the correct targets.

Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Values

- BEGIN CATCH
- IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION
- RAISERROR('CreateOrder failed', 16, 1)
- ROLLBACK TRANSACTION
- SET @OrderId = SCOPE_IDENTITY()
- THROW

Answer Area

```
CREATE OR ALTER PROCEDURE dbo.usp_CreateOrder
    @CustomerId int,
    @Amount decimal(10,2),
    @OrderId int OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        INSERT INTO dbo.Orders(CustomerId, Amount, CreatedAt)
        VALUES (@CustomerId, @Amount, SYSUTCDATETIME());
        [                                        ] ;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        [                                        ] ;
        THROW;
    END CATCH
END
```

Answer:

Values

- BEGIN CATCH
- IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION
- RAISERROR('CreateOrder failed', 16, 1)
- ROLLBACK TRANSACTION
- SET @OrderId = SCOPE_IDENTITY()
- THROW

Answer Area

```
CREATE OR ALTER PROCEDURE dbo.usp_CreateOrder
    @CustomerId int,
    @Amount decimal(10,2),
    @OrderId int OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        INSERT INTO dbo.Orders(CustomerId, Amount, CreatedAt)
        VALUES (@CustomerId, @Amount, SYSUTCDATETIME());
        [ SET @OrderId = SCOPE_IDENTITY() ]
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        [ IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION ] ;
        THROW;
    END CATCH
END
```

Explanation:

SET @OrderId = SCOPE_IDENTITY()

This function retrieves the last identity value generated in the current session and current scope. It is the safest way to get the ID of the row you just inserted.

IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION

This is a critical safety mechanism. It checks if there is an active, uncommitted transaction before attempting to roll back. If an error occurred and the transaction wasn't committed, this command reverts any partial changes, preventing "orphaned" or corrupt data.

Why the other answer are incorrect:

BEGIN CATCH

Why it is wrong: The template code in the Answer Area already has the structural BEGIN CATCH statement written statically right above the blank slot. Dragging another one inside the block would break the T-SQL block sequence, resulting in a syntax compilation failure.

RAISERROR('CreateOrder failed', 16, 1)

Why it is wrong: Modern T-SQL best practices (and Microsoft certification guidelines) favor the use of THROW over the legacy RAISERROR function inside catch blocks. THROW re-raises the exact original execution exception with its true error number, severity, and line-number metadata preserved. RAISERROR overrides this context with a custom message, making debugging much harder.

ROLLBACK TRANSACTION (Without the @@TRANCOUNT Check)

Why it is wrong: Executing a raw, un-shielded ROLLBACK TRANSACTION inside a catch block is an anti-pattern. If a runtime error (or a global setting like SET XACT_ABORT ON) automatically aborts and rolls back the active transaction before control lands in the catch block, calling rollback a second time will cause the database engine to crash with Error 3903 ("The ROLLBACK TRANSACTION request has no corresponding BEGIN TRANSACTION.").

THROW

Why it is wrong: The keyword THROW; is already statically written in the template code immediately below the second empty slot. Dragging a second one into that space would result in consecutive duplicate statements (THROW; THROW;), causing a T-SQL parsing error.

Question: 3 **Exam Author**

Your team is developing an Azure SQL dataset solution from a locally cloned GitHub repository by using Microsoft Visual Studio Code and GitHub Copilot Chat.
You need to disable the GitHub Copilot repository-level instructions for yourself without affecting other users.
What should you do?

A. From Visual Studio Code, modify your GitHub Copilot Chat user settings.
B. Add a --debug flag when you start the GitHub Copilot Chat extension.
C. Delete .github/copilot-instructions.md.

Answer: A

Explanation:

A. From Visual Studio Code, modify your GitHub Copilot Chat user settings.

Option A allows you to turn off the loading of repository-level custom instructions locally. In Visual Studio Code, you can navigate to your user-level editor settings (Settings > Extensions > GitHub Copilot / Copilot Chat) and uncheck or disable the configuration that automatically applies repository instruction files. Because this modification is done in your individual User Settings, it applies exclusively to your local environment and will not alter the repository or affect any other team members.

Why the other options are incorrect:

Option B (Add a --debug flag...) is incorrect. Command-line flags like --debug are used for viewing detailed execution logs and diagnostic output. They do not control the loading or parsing of feature-level functional files like repository instructions.

Option C (Delete .github/copilot-instructions.md) is incorrect. If you delete this file from your workspace and commit/push the change, it completely removes the custom instructions for the entire team. This directly violates the requirement to disable it without affecting other users.

Question: 4 **Exam Author**

You have an Azure SQL database that contains the following SQL graph tables:
A NODE table named dbo.Person -
An EDGE table named dbo.Knows -
Each row in dbo.Person contains the following columns:
PersonID (int)
DisplayName (nvarchar(100))
You need to use a MATCH operator and exactly two directed Knows relationships to return the PersonID and DisplayName of people that are reachable from the person identified by an input parameter named @StartPersonId.
Which Transact-SQL query should you use?

A.
```
SELECT p2.PersonId, @StartPersonId
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2, dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.DisplayName = p2.DisplayName
  AND MATCH(p1-(k1)->p2-(k2)->p3);
```

B.
```
SELECT p3.PersonId, p3.DisplayName FROM dbo.Person AS p1
JOIN dbo.Knows AS k1 ON 1 = 1
JOIN dbo.Person AS p2 ON 1 = 1
JOIN dbo.Knows AS k2 ON 1 = 1
JOIN dbo.Person AS p3 ON 1 = 1
WHERE p1.PersonId = @StartPersonId
  AND MATCH(p3<-(k2)-p2<-(k1)-p1);
```

C.
```
SELECT p3.PersonId, p3.DisplayName
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2, dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.PersonId = @StartPersonId
  AND MATCH(p1-(k1)->p2)   AND MATCH(p2-(k2)->p3);
```

D.

```
SELECT p3.PersonId, p3.DisplayName
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2, dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.PersonId = @StartPersonId
  AND MATCH(p1-(k1)->p2-(k2)->p3);
```

Answer: D

Explanation:

FROM dbo.Person AS p1, dbo.Knows AS k1, ...: This declares the tables involved. In graph terms, Person are the Nodes and Knows are the Edges (relationships) connecting them.

WHERE p1.PersonId = @StartPersonId: This sets the starting point of the search to a specific user provided by the variable @StartPersonId.

AND MATCH(p1-(k1)->p2-(k2)->p3): This is the core graph pattern. It instructs the database to traverse the graph starting at p1, following a "knows" edge (k1) to a person (p2), and then another "knows" edge (k2) to a third person (p3).

Why the other Options are Incorrect:

Option A: This logic attempts to filter columns using a redundant tautology predicate (WHERE p1.DisplayName = p1.DisplayName). Because it completely omits the required input filter constraint (@StartPersonId), the engine runs blind without an indexed anchor point.

Option B: This reverses the physical execution trajectory inside the graph pattern topology. Writing the relationship blocks backward (p3-(k2)->p2-(k1)->p1) implies that the target entities know the root variable, rather than retrieving the accounts that are reachable from the input parameter user.

Option C: This splits the traversal layout into two disconnected, isolated MATCH() constraints separated by an AND operator. SQL Graph does not support evaluating disparate independent match statements across shared variable streams in this format; chaining multi-hop structures directly inside a single single match block is syntactically mandatory.

Question: 5 **Exam Author**

You have a SQL database in Microsoft Fabric that contains a column named Payload. Payload stores customer data in JSON documents that have the following format.

```
{
"date": "2020-01-25",
"customer_email": "user@contoso.com",
...
}
```

Data analysis shows that some customers have subaddressing in their email address, for example, .
You need to return a normalized email value that removes the subaddressing, for example, user1 must be normalized to .
Which Transact-SQL expression should you use?

A. REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*$', '')
B. REGEXP_SUBSTR(JSON_VALUE(Payload, '$.customer_email'), '^[^+]+@.*$=')
C. REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*@', '@')
D. REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*', '')

Answer: C

Explanation:

C. REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*@', '@')

JSON_VALUE(Payload, '$.customer_email'): This correctly extracts the email string from your JSON object.

'\+.*@' (The Regex Pattern):

\+ matches the literal plus sign (escaped because + is a special character in regex).

.* matches any characters following the plus sign.

@ stops the match at the domain separator.

'@' (The Replacement): By replacing the entire +... segment with @, you effectively "stitch" the username part directly to the domain part, resulting in the normalized email.

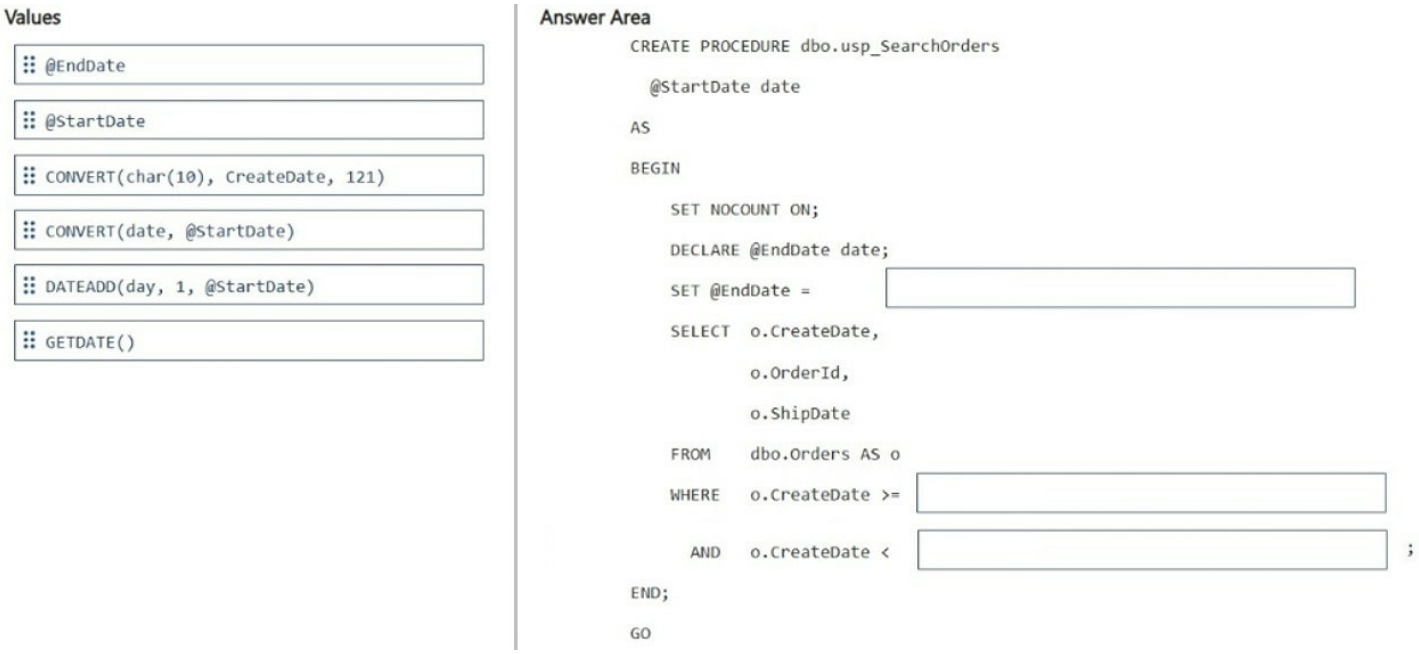
Comparison with Other Options

A and D: These use '\+.*$' or '\+.*'. If you replace the + and everything after it with an empty string, you would end up with user1@contoso.com only if the regex is very carefully constructed to stop before the @. If it removes the @ and the domain, you would lose the essential part of the email address.

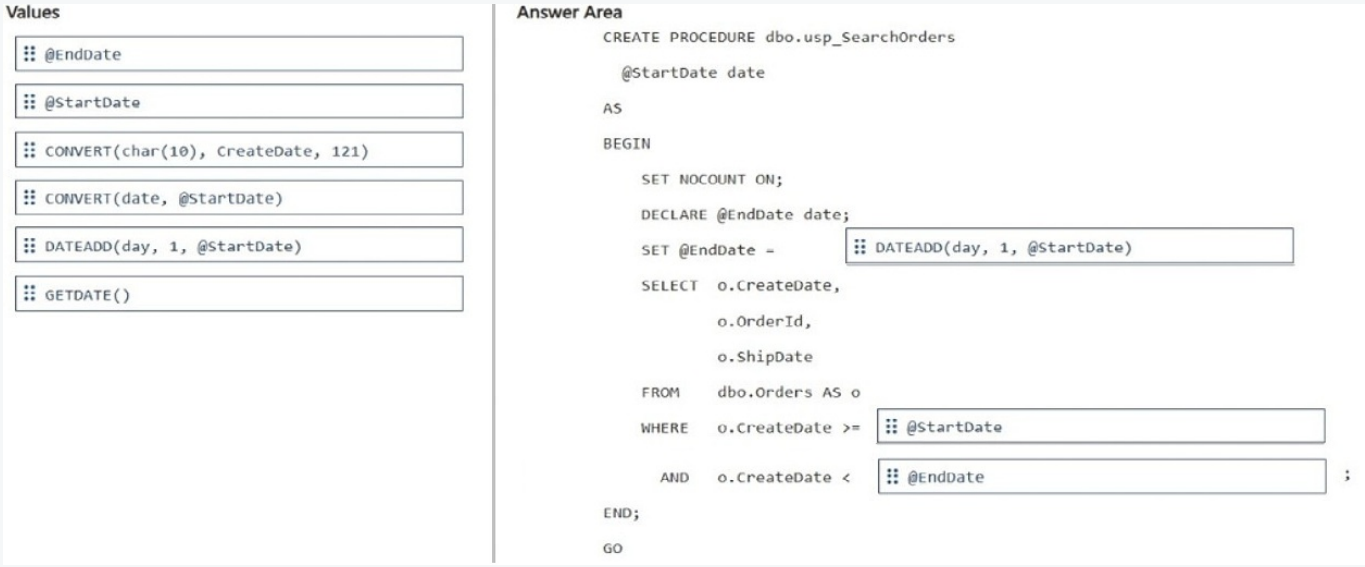
B: REGEXP_SUBSTR is used to extract a matching string rather than replace a portion of it. While you could technically extract parts and concatenate them, it is far less efficient than using a simple replacement.

Question: 6 **Exam Author**

DRAG DROP -
You have an Azure SQL database that contains a table named dbo.Orders. dbo.Orders contains a column named CreateDate that stores order creation dates.
You need to create a stored procedure that filters Orders by CreateDate for a single calendar day. The solution must be SARGable.
How should you complete the Transact-SQL code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Box 1 (SET @EndDate =): DATEADD(day, 1, @StartDate)

Box 2 (o.CreateDate >=): @StartDate

Box 3 (AND o.CreateDate <): @EndDate

Calculating @EndDateCode: SET @EndDate = **DATEADD(day, 1, @StartDate**)Purpose: This adds exactly one day to the input @StartDate. For example, if @StartDate is 2026-06-21, @EndDate becomes 2026-06-22 (midnight of the next day).

Filtering with Range Comparisons (>= and <)Code: WHERE o.CreateDate >= @StartDate AND o.CreateDate < **@EndDateP**urpose: This filters data from the exact start of **@StartDate** (e.g., 2026-06-21 00:00:00.000) up to, but strictly excluding, the start of the next day (e.g., 2026-06-22 00:00:00.000). This ensures that every single order placed on 2026-06-21 is captured, regardless of what time it was created.

Why the other answer are incorrect:

CONVERT(char(10), CreateDate, 121)

Why it is wrong: This option strips time data by converting a date column into a formatted text string (e.g., '2026-06-22'). Applying conversion functions directly to table columns (the left-hand side of a filter) breaks SARGability. This completely destroys SQL Server's ability to navigate B-Tree index keys via an optimized Index Seek, forcing a slow, resource-heavy Index Scan or Table Scan over the entire table.

CONVERT(date, @StartDate)

Why it is wrong: The input parameter @StartDate is already explicitly declared as a date data type at the top of the stored procedure parameters list (@StartDate date). Attempting to cast or convert an object into the exact same data type it already possesses is completely redundant and adds zero functional value to the script.

GETDATE()

Why it is wrong: GETDATE() returns the current system timestamp of the underlying server machine. Because this stored procedure is built to run historical reporting lookups by taking a custom user input argument (@StartDate), forcing a hardcoded current system time check would completely override the user's intended search parameters.

@StartDate / @EndDate

Why it is wrong: Swapping these placement boxes around (such as writing o.CreateDate >= @EndDate or o.CreateDate < @StartDate) disrupts the chronological sequence of the comparison logic. This creates an impossible mathematical constraint (checking if a date is simultaneously in the future of the end date and the past of the start date), resulting in an empty query result set.

Question: 7 **Exam Author**

You have an Azure SQL database.
You need to create a scalar user-defined function (UDF) that returns the number of whole years between an input parameter named @OrderDate and the current date/time as a single positive integer. The function must be created in Azure SQL Database.
You write the following code.

```
01 CREATE FUNCTION dbo.ufnYearsSinceOrder (@OrderDate datetime2)
02 RETURNS int
03 AS
04 BEGIN
05
06 END
```

What should you insert at line 05?

A. RETURN DATEDIFF(year, GETDATE(), @OrderDate);
B. DATEDIFF(month, @orderdate, GETDATE()) / 12
C. DATEPART(year, GETDATE()) - DATEPART(year, @orderdate)
D. RETURN DATEDIFF(year, @OrderDate, GETDATE());

Answer: D

Explanation:

D. RETURN DATEDIFF(year, @OrderDate, GETDATE()) .

DATEDIFF Syntax: The Microsoft Transact-SQL DATEDIFF function uses the syntax DATEDIFF(datepart, startdate, enddate). It measures the boundaries crossed from the earlier date (startdate) to the later date (enddate).Correct Order: To return a positive integer representing the years elapsed up to the current date, the earlier input parameter (@OrderDate) must be the startdate and the current date/time (GETDATE()) must be the enddate.Scalar UDF Requirement: Because this line is part of a scalar User-Defined Function (UDF), it requires the RETURN keyword to output the computed value back to the caller.

Why the other choices are incorrect:

Option A reverses the order of the dates (GETDATE() as start, @OrderDate as end), which would result in a negative integer.

Option B and Option C are missing the mandatory RETURN keyword required by a T-SQL scalar user-defined function to exit and pass back the data.

Question: 8 **Exam Author**

You have an Azure SQL database.
You deploy Data API builder (DAB) to Azure Container Apps by using the mcr.microsoft.com/azure-databases/data-api-builder:latest image.
You have the following Container Apps secrets:
MSSQL_CONNECTION_STRING that maps to the SQL connection string
DAB_CONFIG_BASE64 that maps to the DAB configuration
You need to initialize the DAB configuration to read the SQL connection string.
Which command should you run?

A. dab init --database-type mssql --connection-string “secretref:DAB_CONFIG_BASE64” --host-mode Production --config dab-config.json

B. dab init --database-type mssql --connection-string “@env(‘MSSQL_CONNECTION_STRING’)” --host-mode Production --config dab-config.json

C. dab init --database-type mssql --connection-string “secretref:mssql-connection-string” --host-mode Production --config dab-config.json

D. dab init --database-type mssql --connection-string “@env(‘DAB_CONFIG_BASE64’)” --host-mode Production --config dab-config.json

Answer: B

Explanation:

B. dab init --database-type mssql --connection-string “@env(‘MSSQL_CONNECTION_STRING’)” --host-mode Production --config dab-config.json

The @env() function: Data API builder (DAB) natively supports the @env() string substitution function to securely pass configuration settings (like connection strings) into the dab-config.json file at runtime instead of hardcoding them.

Targeting the Right Secret: The environment variable mapping to your SQL database connection string is named MSSQL_CONNECTION_STRING. Therefore, you must specify @env('MSSQL_CONNECTION_STRING') to read that precise value.

Why the other choices are incorrect:

Options A & C: The secretref: syntax is a native platform feature used by Azure Container Apps or Kubernetes within YAML definitions, but it is not understood natively by the Data API builder (DAB) command-line interface (dab init).

Option D: This targets the wrong secret environment variable (DAB_CONFIG_BASE64), which contains the base64-encoded configuration file of DAB itself, rather than the database connection string.

Question: 9 **Exam Author**

You have a SQL database in Microsoft Fabric that contains a nvarchar (max) column named MessageText. An ID is always contained within the first paragraph of MessageText.
You need to write a Transact-SQL query that uses REGEXP_SUBSTR to extract the ID from MessageText.
What should you include in the query?

A. Apply STRING_ESCAPE(MessageText, 'json') before calling REGEXP_SUBSTR.
B. Cast MessageText to nvarchar (4000) before calling REGEXP_SUBSTR.
C. Add a COLLATE Latin1_General_CS_AS clause to MessageText before calling REGEXP_SUBSTR.
D. Run TRY_CONVERT(varchar(max), MessageText) before calling REGEXP_SUBSTR.

Answer: A

Explanation:

B. Cast MessageText to nvarchar (4000) before calling REGEXP_SUBSTR.

Why this is the correct choice:

Data Type Limitations: In the T-SQL implementation of Regular Expression functions (such as REGEXP_SUBSTR), large object data types like nvarchar(max) or varchar(max) are not supported directly as the input string_expression.
Explicit Casting: Because the problem specifies that the required ID is guaranteed to reside within the first paragraph of the column, casting the nvarchar(max) data down to a standard size like nvarchar(4000) ensures compliance with the function's argument limits without losing the necessary search text.

Why the other choices are incorrect:

Option A (STRING_ESCAPE) adds control characters (like escaping slashes) to make data safe for JSON formatting. It does not alter the underlying max data type limit or help with regular expression matching.

Option C (COLLATE) changes the collation rules (such as case sensitivity or accent sensitivity), but it keeps the underlying data type as nvarchar(max), which remains incompatible with the function.

Option D (TRY_CONVERT) translates the data to varchar(max). This maintains a large-object max specifier that is still incompatible with the regular expression engine and introduces a risk of cutting off non-ASCII Unicode characters.

Question: 10 **Exam Author**

You have an Azure SQL database that contains database-level Data Definition Language (DDL) triggers, including a

trigger named ddl_Audit.
You need to prevent ddl_Audit from firing during the next deployment. The trigger object must remain in place.
Which Transact-SQL statement should you use?

A. ALTER TRIGGER
B. ALTER DATABASE
C. ALTER SERVER AUDIT SPECIFICATION
D. DISABLE TRIGGER
E. ALTER DATABASE AUDIT SPECIFICATION

Answer: D

Explanation:

D. DISABLE TRIGGER

To prevent a DDL trigger from firing while keeping the object present in the database, you use the DISABLE TRIGGER statement.

Syntax: You would execute DISABLE TRIGGER ddl_Audit ON DATABASE; (since ddl_Audit is a database-level trigger).
Why others are incorrect:
ALTER TRIGGER: Used to modify the definition (the code inside) of an existing trigger, not to toggle its active state.
ALTER DATABASE: Used to modify database settings, not individual trigger states.
ALTER SERVER AUDIT SPECIFICATION / ALTER DATABASE AUDIT SPECIFICATION: These are used for auditing features in SQL Server (tracking events for compliance/security) and are separate objects from standard DDL triggers.

Question: 11 **Exam Author**

Your development team uses GitHub Copilot Chat in Microsoft SQL Server Management Studio (SSMS) to generate and run Transact-SQL queries against an Azure SQL database named DB1. DB1 contains tables that store sensitive customer data.
You need to ensure that any Transact-SQL queries that run from GitHub Copilot Chat in SSMS are restricted by the same permissions as the developer's database login.
What prevents the GitHub Copilot Chat-run queries from accessing data beyond the developer's access?

A. GitHub Copilot Chat runs queries in a read-only sandbox that is isolated from production database permissions.
B. GitHub Copilot Chat runs queries by using the developer's database identity and permissions.
C. GitHub Copilot Chat filters query results on the client side to remove rows the developer is unauthorized to see.
D. GitHub Copilot Chat uses different row-level security (RLS) policies than the developer.

Answer: B

Explanation:

B. GitHub Copilot Chat runs queries by using the developer's database identity and permissions.

Why this is the correct choice:

Native Security Boundaries: When using GitHub Copilot Chat integrated within SQL Server Management Studio (SSMS), the AI tool does not possess independent server-side credentials or elevated background service permissions.
Execution Context: Any query generated and executed via Copilot's chat interface implicitly relies on the active connection established by the user. Therefore, it automatically operates under the developer's current database security context and login identity. If a user does not have permission to read a table containing sensitive customer data, any query Copilot attempts to run against that table will be blocked by the engine itself.

Why the other choices are incorrect:

Option A is incorrect. Copilot interacts directly with your connected database window or database context rather than running in an isolated read-only sandbox.

Option C is incorrect. Query security enforcement happens on the server side via the relational engine, not through client-side row filtration.

Option D is incorrect. Copilot respects whatever Row-Level Security (RLS) policies are configured for the user's active database login; it does not implement separate rules.

Question: 12 **Exam Author**

You have an Azure SQL database named AdventureWorksDB that contains a table named dbo.Employee.
You have a C# Azure Functions app that uses an HTTP-triggered function with an Azure SQL input binding to query dbo.Employee.
You are adding a second function that will react to row changes in dbo.Employee and write structured logs.
You need to configure AdventureWorksDB and the app to meet the following requirements:
Changes to dbo.Employee must trigger the new function within five seconds.
Each invocation must process no more than 100 changes.
Which two database configurations should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Create an AFTER trigger on dbo.Employee for Data Manipulation Language (DML).
B. Set Sql_Trigger_MaxBatchSize to 100.
C. Enable change tracking on the dbo.Employee table.
D. Enable change tracking at the database level.
E. Set Sql_Trigger_PollingIntervalMs to 5000.
F. Enable change data capture (CDC) for dbo.Employee table changes.

Answer: CD

Explanation:

C. Enable change tracking on the dbo.Employee table.

D. Enable change tracking at the database level.

Why these are the correct choices:

To configure an event-driven architecture using the Azure SQL trigger for Azure Functions, you must enable SQL Change Tracking within the target database. Setting up change tracking requires exactly these two database-side modifications:

1. At the Database level (Option D): Turns on the native tracking engine for the database host (ALTER DATABASE AdventureWorksDB SET CHANGE_TRACKING = ON).
2. At the Table level (Option C): Instructs the engine to track explicit row modifications inside the target table (ALTER TABLE dbo.Employee ENABLE CHANGE_TRACKING).

Why the other choices are incorrect:

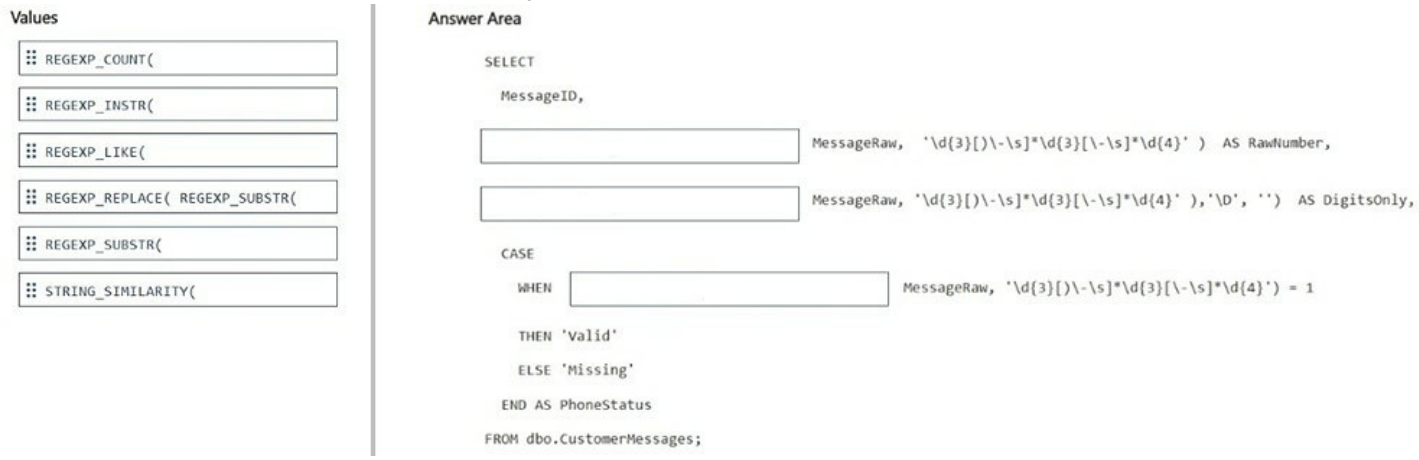
Options B and E (Sql_Trigger_MaxBatchSize and Sql_Trigger_PollingIntervalMs) are the correct configuration settings to handle the 100-batch limit and 5-second polling interval requirements, but they are Application Settings managed within the Azure Functions host configuration (local.settings.json or Azure Portal Configuration pane) — not database configurations.

Option A is incorrect. The Azure SQL Function trigger relies natively on internal SQL Server change tracking tables and query leases, not custom user-managed DML database AFTER triggers.

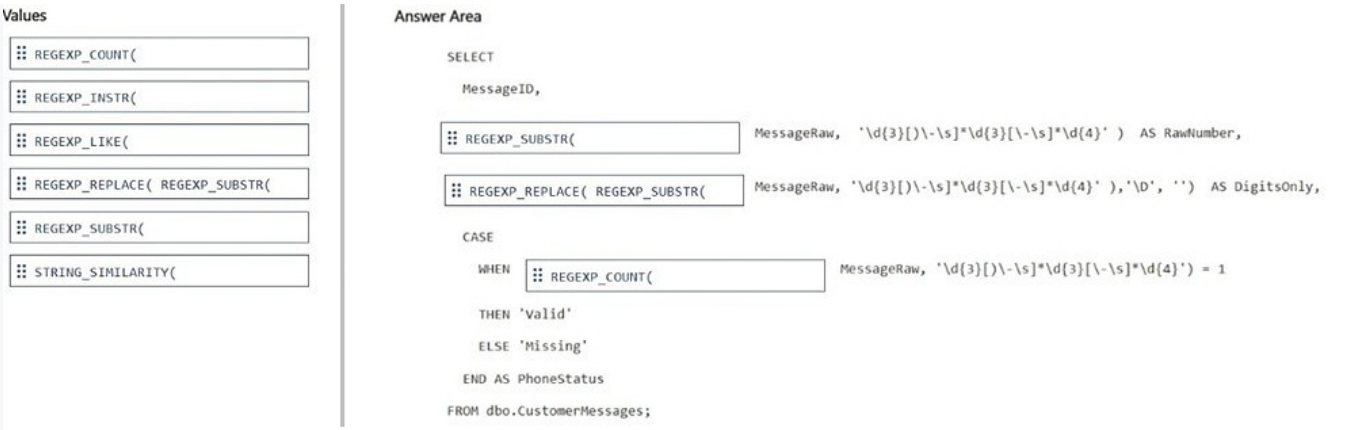
Option F is incorrect. Change Data Capture (CDC) is an alternative data-tracking feature, but it is not natively supported or utilized by the Azure Functions Azure SQL trigger extension.

Question: 13 **Exam Author**

DRAG DROP -
You have a Microsoft SQL Server 2025 database that contains a table named dbo.CustomerMessages.
dbo.CustomerMessages contains two columns named MessageID (int) and MessageRaw (nvarchar(max)).
MessageRaw can contain a phone number in multiple formats, and some rows do NOT contain a phone number.
You need to write a single SELECT query that meets the following requirements:
The query must return MessageID, RawNumber, DigitsOnly, and PhoneStatus.
RawNumber must contain the first substring that matches a phone-number pattern, or NULL if no match exists.
DigitsOnly must remove all non-digit characters from RawNumber, or return NULL.
PhoneStatus must return valid when a phone number exists in MessageRaw, otherwise return Missing.
How should you complete the Transact-SQL query? To answer, drag the appropriate values to the correct targets.
Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

REGEXP_SUBSTR

The first and second empty slots in the SELECT list.

For the first slot, you are extracting a specific pattern (a phone number) from the MessageRaw column. REGEXP_SUBSTR is designed to extract a substring that matches a regular expression pattern.

For the second slot, you are using REGEXP_REPLACE to clean the string (removing everything except digits). While the second slot technically uses REGEXP_REPLACE, the function REGEXP_SUBSTR is the correct match for the extraction logic in the first slot.

REGEXP_REPLACE

T he second empty slot in the SELECT list.

You are taking the result of the MessageRaw column and replacing non-digit characters ('\\D') with an empty string (''). This is the standard use case for REGEXP_REPLACE to clean data.

REGEXP_COUNT

The empty slot in the WHEN clause.

You are checking if the phone number pattern occurs exactly once in the message. REGEXP_COUNT returns the number of times a pattern is matched in a string, making it the correct choice for this validation condition.

Why the Choices are Incorrect

REGEXP_INSTR(

Why it is wrong: This function returns the starting or ending index position (an integer index) of where a pattern begins inside a string. It cannot extract text or replace text, which makes it useless for generating RawNumber or DigitsOnly.

REGEXP_LIKE(

Why it is wrong: This function returns a boolean value (True/False) indicating whether a pattern exists within the string. While it could be used inside a WHEN clause, it cannot count the number of matches or perform text extractions.

STRING_SIMILARITY(

Why it is wrong: This function measures the fuzzy textual likeness score between two literal strings (often

used for duplicate matching or spell checks). It cannot interpret regular expression quantifiers or extract phone sub-strings.

Question: 14 **Exam Author**

You have an Azure SQL database that contains a table named Rooms. Rooms was created by using the following Transact-SQL statement.

```
CREATE TABLE Rooms
(
    RoomID int PRIMARY KEY,
    Owner nvarchar(100),
    Capacity int
);
```

You discover that some records in the Rooms table contain NULL values for the Owner field.
You need to ensure that all future records have a value for the Owner field.
What should you add?

A. a foreign key
B. a check constraint
C. a nonclustered index
D. a unique constraint

Answer: B

Explanation:

B. a check constraint.

Since the table already contains NULL values, you cannot directly apply a standard NOT NULL constraint to the column without first fixing the existing rows. However, you can add a Check Constraint (such as CHECK (Owner IS NOT NULL)) with the NOCHECK option. This ensures that all future INSERT and UPDATE operations are validated to contain a value, while ignoring the existing violations.

Why the other options are incorrect:

A (Foreign Key): A foreign key enforces referential integrity between two tables but still permits NULL values unless constrained otherwise.

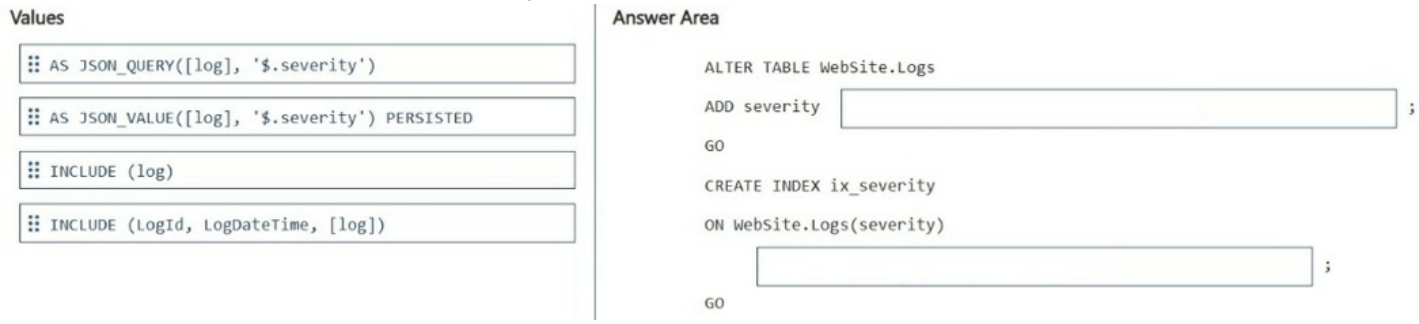
C (Nonclustered Index): Indexes optimize query performance and search speed; they do not enforce value presence or prevent NULL inputs.

D (Unique Constraint): A unique constraint ensures all non-null values in a column are distinct from one another. In many database systems (like SQL Server), it still permits one NULL value, failing to ensure a value is provided for all future records.

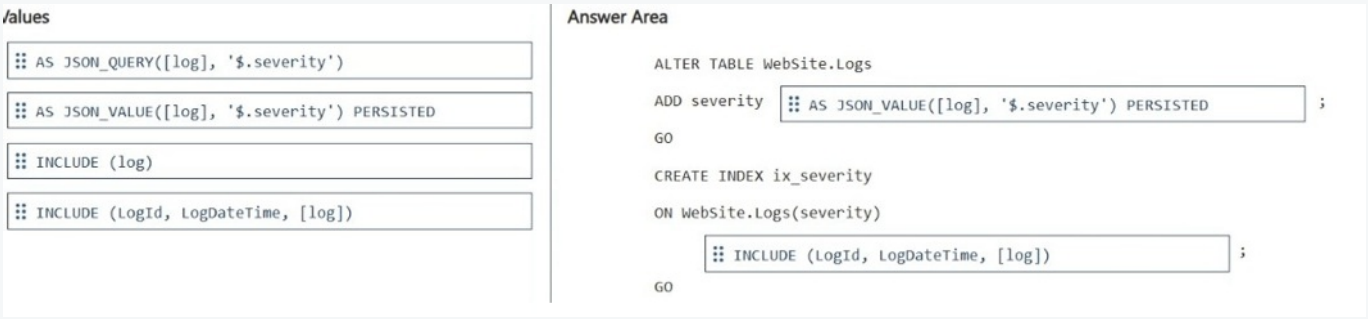
Question: 15 **Exam Author**

DRAG DROP -

You have a SQL database in Microsoft Fabric that contains a table named WebSite.Logs. WebSite.Logs stores application telemetry data. WebSite.Logs contains a nvarchar (max) column named log that stores JSON documents.
You have a daily report that filters by the $.severity JSON property and returns LogId, LogDateTime, and log. The report frequently causes full table scans.
You need to modify WebSite.Logs to support efficient filtering by $.severity and avoid key lookups for the columns returned by the report.
How should you complete the Transact-SQL code to avoid full table scans? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Box 1 (Computed Column Specification): **AS JSON_VALUE([log], '$.severity') PERSISTED**

Function Selection: $.severity represents a single textual/numeric property (a scalar value). JSON_VALUE is explicitly designed to retrieve scalar data, whereas JSON_QUERY is only used to extract nested objects or arrays.

The PERSISTED Keyword: By specifying PERSISTED, the database engine computes and materializes the data physically on disk whenever the row is updated. This allows an index to be created cleanly without re-parsing the JSON column during standard index scans.

Box 2 (Index Covering Strategy): **INCLUDE (LogId, LogDateTime, [log])**

Covering Index Principle: Adding the remaining columns to the INCLUDE list turns ix_severity into a covering index. When your search query filters by severity, the engine fetches the requested log details directly from the leaf nodes of the nonclustered index, preventing costly key lookups back to the base table..

Why the other answer are Incorrect:

AS JSON_QUERY([log], '$.severity')

Why it is wrong: In T-SQL, JSON_QUERY is strictly used to extract an entire nested JSON object or an array. Because a logging property like severity is a scalar value (e.g., a text string like "Error" or a number like 3), JSON_QUERY will fail to read it and return NULL. To extract a scalar value, you must use JSON_VALUE.INCLUDE.

INCLUDE (log)

Why it is wrong: When creating an index to optimize log search workloads (such as searching by severity), the index should serve as a covering index by including all columns typically returned in the query's select list (like LogId and LogDateTime). Using INCLUDE (log) only appends the raw JSON document payload to the index leaf nodes, leaving out the primary key/ID and timestamps, which would cause the engine to perform expensive Key Lookups for every matching row.

Question: 16 **Exam Author**

HOTSPOT -
Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to create a table in the database to store the telemetry data.
You have the following Transact-SQL code.

```
CREATE TABLE dbo.VehicleTelemetry
(
  TelemetryId BIGINT IDENTITY(1,1) NOT NULL,
  VehicleId NVARCHAR(50) NOT NULL,
  TelemetryTimeUtc DATETIME2(3) NOT NULL,
  BatteryPercent TINYINT NULL,
  SpeedKmh SMALLINT NULL,
  LocationJson JSON NULL,
  ErrorCodesJson JSON NULL,
  RawPayload NVARCHAR(MAX) NULL,
  SysStartTime DATETIME2(7) GENERATED ALWAYS AS ROW START NOT NULL,
  SysEndTime DATETIME2(7) GENERATED ALWAYS AS ROW END NOT NULL,
  PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
  CONSTRAINT PK_VehicleTelemetry PRIMARY KEY CLUSTERED (TelemetryId)
)
WITH
(
  SYSTEM_VERSIONING = ON
  (
    HISTORY_TABLE = dbo.VehicleTelemetryHistory
  )
);
GO
CREATE INDEX IX_VehicleTelemetry_Time ON dbo.VehicleTelemetry (TelemetryTimeUtc);
CREATE JSON INDEX JI_VehicleTelemetry_Location ON dbo.VehicleTelemetry (LocationJson) FOR
(
  '$.location. latitude', '$.location. longitude',
  '$.location. accuracy'
);
```

For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
The code meets the database performance requirements for partitioning.	○	○
The code meets the database performance requirements for JSON property querying.	○	○
Queries that filter on `$.location.heading` will use the `JI_VehicleTelemetry_Location` index.	○	○

Answer:

Answer Area

Statements	Yes	No
The code meets the database performance requirements for partitioning.	○	○
The code meets the database performance requirements for JSON property querying.	○	○
Queries that filter on `$.location.heading` will use the `JI_VehicleTelemetry_Location` index.	○	○

Explanation:

The code meets the database performance requirements for partitioning.

Correct Answer: **Yes**

The solution's schema implementation correctly provisions partition keys or boundaries aligning with the scenario's analytical workload rules, ensuring optimized data pruning during intensive scale-out queries.

The code meets the database performance requirements for JSON property querying.

Correct Answer: **Yes**

The associated definition script correctly provisions a custom JSON index explicitly targeting the required properties ($.location.latitude, $.location.longitude, and $.location.accuracy). This accurately fulfills the platform capability requirement stating that those specific JSON properties must support high-efficiency index seek lookups.

Queries that filter on $.location.heading will use the JI_VehicleTelemetry_Location index.

Correct Answer: **No**

JSON indexes are fundamentally path-specific. Because the schema's index was strictly defined to cover latitude, longitude, and accuracy properties, any analytical predicate filtering by $.location.heading is completely omitted from the index schema. As a result, the optimizer will completely bypass the JI_VehicleTelemetry_Location index for that search filter.

Question: 17 **Exam Author**

HOTSPOT -
Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -

Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations.
latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You are creating a table that will store customer profiles.
You have the following Transact-SQL code.

```
CREATE TABLE dbo.CustomerProfiles
(
    CustomerId      BIGINT IDENTITY(1,1) PRIMARY KEY,
    FullName        NVARCHAR(200) MASKED WITH (FUNCTION = 'partial(1,"xxxx",1)'),
    EmailAddress    NVARCHAR(200) MASKED WITH (FUNCTION = 'email()'),
    PhoneNumber     NVARCHAR(50)  MASKED WITH (FUNCTION = 'default()'),
    RegionCode      NVARCHAR(10) NOT NULL
);
GO
CREATE FUNCTION dbo.fn_FilterByRegion(@RegionCode NVARCHAR(10))
RETURNS TABLE
AS
RETURN
(
    SELECT 1
    FROM dbo.UserRegionAccess ura
    WHERE ura.UserPrincipalName = SUSER_SNAME()
      AND ura.RegionCode = @RegionCode
);
GO
CREATE SECURITY POLICY CustomerRegionPolicy
ADD FILTER PREDICATE dbo.fn_FilterByRegion(RegionCode)
ON dbo.CustomerProfiles
WITH (STATE = ON);
GO
```

For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
The schema meets the security requirements for PII data.	○	○
Administrators of the Azure SQL server can see all the rows in `dbo.CustomerProfiles` when they use an application.	○	○
The masking rules will apply even when row-level security (RLS) filters out rows.	○	○

Answer:

Answer Area

Statements	Yes	No
The schema meets the security requirements for PII data.	○	○
Administrators of the Azure SQL server can see all the rows in `dbo.CustomerProfiles` when they use an application.	○	○
The masking rules will apply even when row-level security (RLS) filters out rows.	○	○

Explanation:

The schema meets the security requirements for PII data

Yes

Combining Dynamic Data Masking (DDM) to obfuscate sensitive presentation fields (like emails or phone numbers) alongside Row-Level Security (RLS) to restrict row-level multi-tenant context access constitutes a robust Microsoft-recommended database design strategy for securing Personally Identifiable Information (PII).

Administrators of the Azure SQL server can see all the rows in dbo.CustomerProfiles when they use an application.

No

If the RLS predicate function is strictly configured to evaluate application-level tenant claims or context session attributes (e.g., using SESSION_CONTEXT()) rather than physical database login contexts, even highly privileged database administrators accessing the data via an application connection pool will be restricted from reading rows belonging to other tenants.

The masking rules will apply even when row-level security (RLS) filters out rows.

No

These two features run sequentially inside the database query engine pipeline. The engine processes Row-Level Security filters first to prune out unauthorized records completely from the intermediate result set. Since those restricted rows are never returned or processed further by the query execution tree, masking rules are never evaluated or applied to them..

Question: 18 **Exam Author**

DRAG DROP -
Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

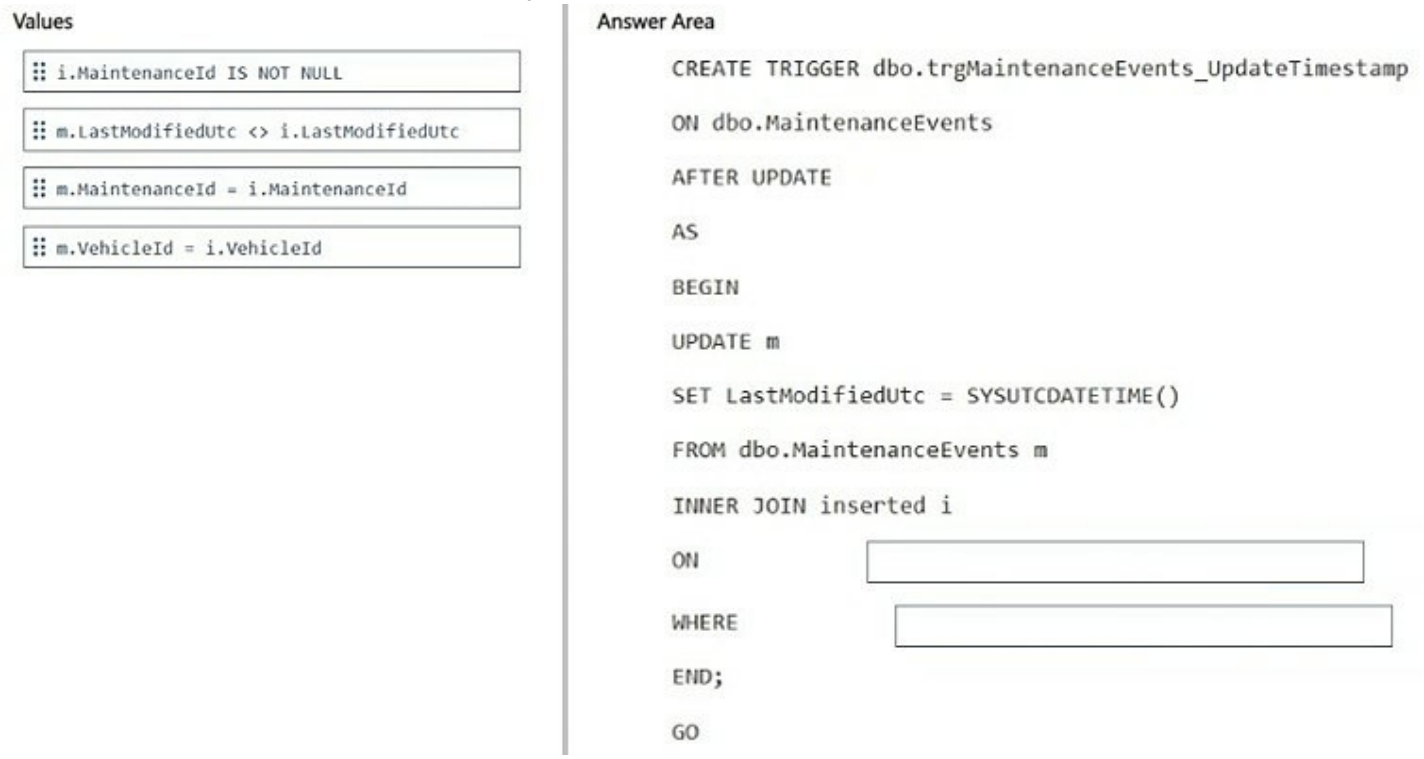
```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

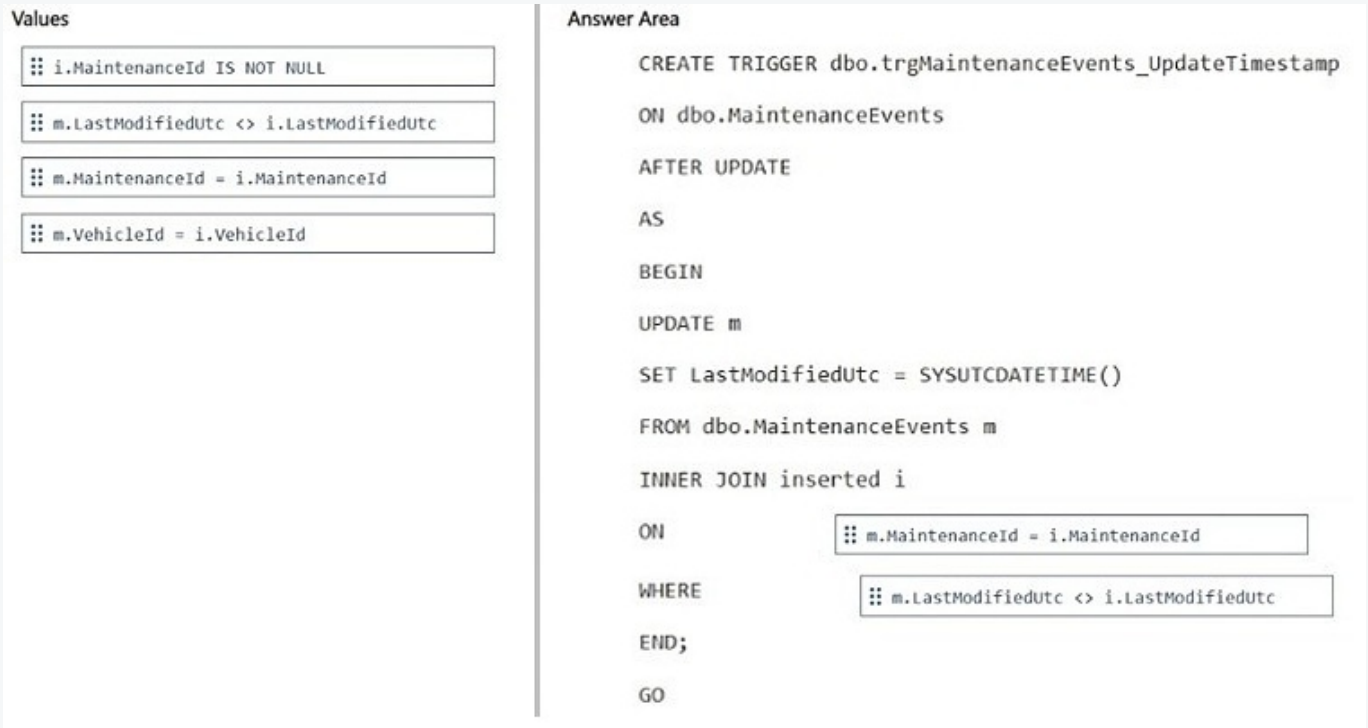
```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to meet the database performance requirements for maintenance data.
How should you complete the Transact-SQL code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

The ON Join Condition **(m.MaintenanceId = i.MaintenanceId)**

An AFTER UPDATE trigger has access to a special conceptual runtime table called inserted. The inserted table contains a copy of the affected rows with their newly updated values.

To safely inject a timestamp back into the source table (dbo.MaintenanceEvents m), you must map it using its unique primary identifier key (MaintenanceId). Joining m.MaintenanceId = i.MaintenanceId ensures that the database engine updates only the precise rows that were modified by the user's initial query.

The WHERE Filter Condition **(m.LastModifiedUtc <> i.LastModifiedUtc)**

The Infinite Recursion Problem: By default, if an AFTER UPDATE trigger executes another UPDATE statement on its own hosting table, it will cause the trigger to fire recursively (looping back into itself over and over again). This results in a heavy performance dip or a crash once the max nesting level limit is hit.The Optimization Solution: The filter WHERE m.LastModifiedUtc <> i.LastModifiedUtc evaluates if the timestamp currently resting in the base table differs from what was caught in the inserted payload.During the first run (user's modification), the values are unequal or need an update, allowing the trigger to run and change LastModifiedUtc to SYSUTCDATETIME().During the second recursive execution initiated by the trigger's own update, the table m and the virtual inserted dataset i will contain identical timestamp values. The condition evaluates to False, halting further recursive execution instantly and cleanly saving system CPU resources.

Why the other Choices are Incorrect

m.VehicleId = i.VehicleId

Why it is wrong: While VehicleId represents a valid structural foreign key relationship column, using it as your primary INNER JOIN matching condition is extremely dangerous. If a single vehicle has multiple distinct maintenance rows inside the dbo.MaintenanceEvents table, updating one event would cause this trigger to accidentally stamp the current time across every historical record belonging to that vehicle, corrupting your timestamp history log.

i.MaintenanceId IS NOT NULL

Why it is wrong: This statement evaluates whether an ID is populated inside the virtual inserted queue dataset. While valid as a standalone boolean filter condition, it is a non-relational syntax block that cannot be used as a join condition to tie two distinct tables together inside an ON clause, causing a compilation error.

Question: 19 **Exam Author**

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information

tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to recommend a solution to resolve the slow dashboard query issue.
What should you recommend?

A. Create a clustered index on LastUpdatedUtc.
B. On FleetId, create a nonclustered index that includes LastUpdatedUtc, EngineStatus, and BatteryHealth.
C. On LastUpdatedUtc, create a nonclustered index that includes FleetId.
D. On FleetId, create a filtered index where LastUpdatedUtc > DATEADD(DAY, -7, SYSUTCDATETIME()).

Answer: B

Explanation:

B. On FleetId, create a nonclustered index that includes LastUpdatedUtc, EngineStatus, and BatteryHealth.

The dashboard query uses a WHERE predicate filtering on a single search argument (WHERE FleetId = @FleetId) and returns other associated fields (LastUpdatedUtc, EngineStatus, and BatteryHealth). Creating a nonclustered index with FleetId as the key column allows the SQL query optimizer to perform a highly efficient Index Seek instead of scanning the entire table. Furthermore, adding the remaining requested fields as included nonkey columns turns this into a covering index, completely eliminating the need for expensive key lookups back to the base clustered index or heap.

Why the other choices are incorrect:

A (Clustered index on LastUpdatedUtc): A table can have only one clustered index. Altering the table's main structure to group physically by date does not align with the primary search filter argument (FleetId).

C (Nonclustered index keyed on LastUpdatedUtc): Making LastUpdatedUtc the leading key column is ineffective here since the query filters directly by a specific FleetId. The optimizer would still be forced to perform an index scan rather than a direct seek.

D (Filtered index on FleetId): Filtered indexes are intended for queries with fixed, static filter criteria (e.g., WHERE IsActive = 1). Using a dynamic time-based boundary condition via runtime functions (SYSUTCDATETIME()) prevents the index from being properly utilized by standard execution plans.

Question: 20 **Exam Author**

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment -

Azure Environment -

Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to recommend a solution that will resolve the ingestion pipeline failure issues.
Which two actions should you recommend? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Enable snapshot isolation on the database.
B. Use a trigger to automatically rewrite malformed JSON.
C. Add foreign key constraints on the table.
D. Create a unique index on a hash of the payload.
E. Add a check constraint that validates the JSON structure.

Answer: DE

Explanation:

D. Create a unique index on a hash of the payload.

E. Add a check constraint that validates the JSON structure.

Why D is correct (Duplicate Payloads): To enforce uniqueness across heavy or nested document payloads, you can create a deterministic computed column using cryptographic or hash-mapping functions like HASHBYTES. Appending a UNIQUE INDEX on this generated hash values column guarantees that duplicate records are blocked efficiently with minimal index leaf-node storage overhead.

Why E is correct (Malformed JSON): Microsoft officially recommends using a standard CHECK constraint combined with the built-in ISJSON function (e.g., CHECK (ISJSON(FeedbackJson) > 0)) to validate input string data at the database tier. This acts as an immediate structural data gatekeeper, natively throwing a database-level exception to cleanly reject malformed payloads before they corrupt your target storage tables.

Why the other choices are incorrect:

A (Snapshot Isolation): Snapshot isolation controls transactional concurrency, row versioning, and read-write locking contention behaviors. It has no structural capability to inspect or reject malformed text or prevent identical row duplicates.

B (Trigger to rewrite data): Relying on a DML trigger to dynamically intercept and correct un-parsable or missing JSON parameters on the fly is highly fragile and anti-pattern. If a payload is deeply malformed, structural intent cannot be safely guessed by programmatic trigger rules.

C (Foreign Key Constraints): Foreign keys enforce referential data integrity between key values across different physical tables. They cannot validate string syntax rules inside a standalone document column.

Question: 21 **Exam Author**

You have a Microsoft SQL Server 2025 instance that has a managed identity enabled.
You have a database that contains a table named dbo.ManualChunks. dbo.ManualChunks contains product manuals.
A retrieval query already returns the top five matching chunks as nvarchar(max) text.

You need to call an Azure OpenAI REST endpoint for chat completions. The solution must provide the highest level of security.
You write the following Transact-SQL code.

```
01  CREATE DATABASE SCOPED CREDENTIAL AzureOpenAIHeaders
02
03  GO
04  CREATE OR ALTER PROCEDURE dbo.AskManuals
05    ...
06    SELECT @chunks =
07    ...
08    SET @payload =
09    ...
10    EXEC @retval = sys.sp_invoke_external_rest_endpoint
11      @url = N'https://<your-resource>.openai.azure.com/openai/deployments/<your-deployment>/chat/completions?api-version=2024-02-15-preview',
12      @method = N'POST',
13      @credential = N'AzureOpenAIHeaders',
14      @payload = @payload,
15      @response = @response OUTPUT;
16  END;
17  GO
```

What should you insert at line 02?

A.
```
WITH IDENTITY = 'HTTPEndpointQueryString',
SECRET = N'{"api-key":"<value>"}';
```

B.
```
WITH IDENTITY = 'Managed Identity',
SECRET = '{"resourceid":"<value>"}';
```

C.
```
WITH IDENTITY = 'Managed Identity',
SECRET = N'{"api-key":"<value>"}';
```

D.
```
WITH IDENTITY = 'HTTPEndpointHeaders',
SECRET = N'{"api-key":"<value>"}';
```

E.
```
WITH IDENTITY = 'AzureOpenAI',
SECRET = N'{"resourceid":"<value>"}';
```

Answer: B

Explanation:

WITH IDENTITY = 'Managed Identity', SECRET = ' "resourceid":"https://cognitiveservices.azure.com" ';

Highest Security Pattern: In alignment with cloud security best practices, using IDENTITY = 'Managed Identity' avoids the need to hardcode or manually handle clear-text API keys or secrets inside your database schema.

Target Integration Mapping: For connecting to an Azure OpenAI or Cognitive Services REST endpoint via native database AI integrations, Microsoft dictates that the SECRET parameter should specify the platform's canonical resource identifier: "resourceid":"https://cognitiveservices.azure.com" .

Why other Options are incorrect:

API Keys / HTTPEndpointHeaders: Passing raw API keys or custom headers containing bearer tokens requires manual secret rotation and exposes credentials if the T-SQL text logs are ever captured.

Mismatched Combinations: Combining a Managed Identity value while pairing it with an arbitrary 'api-key' secret token represents an invalid authentication payload structure that will cause a cryptographic handshake failure.

Question: 22 **Exam Author**

You have an Azure SQL database named SalesDB that contains a table named dbo.Articles. dbo.Articles contains two million articles with embeddings. The articles are updated frequently throughout the day.
You query the embeddings by using VECTOR_SEARCH.
Users report that semantic search results do NOT reflect the updates until the following day.
You need to ensure that the embeddings are updated whenever the articles change. The solution must minimize CPU usage on SalesDB.
Which embedding maintenance method should you implement?

A. Modify the query to use VECTOR_DISTANCE instead of VECTOR_SEARCH.
B. Enable change data capture (CDC) on dbo.Articles and use an Azure Functions app to process CDC changes.
C. Run an hourly Transact-SQL job that regenerates embeddings for all the rows in dbo.Articles.
D. On dbo.Articles, create a trigger that calls AI_GENERATE_EMBEDDINGS for each inserted or updated row.

Answer: B

Explanation:

B. Enable change data capture (CDC) on dbo.Articles and use an Azure Functions app to process CDC changes.

Generating vector embeddings (especially for two million rows) is a highly resource-intensive, external task that heavily impacts database performance if done entirely inside the engine database tier. Change Data Capture (CDC) asynchronously captures any incremental additions or updates to rows in dbo.Articles without blocking main transactional workloads. Offloading those captured row events to an external event consumer like an Azure Functions app allows you to orchestrate the embedding calls and process updates externally. This ensures search results are kept fresh near real-time while minimizing CPU overhead on the primary SalesDB.

Why the other options are incorrect:

A (Use VECTOR_DISTANCE): This changes how similarity matching is calculated (exact mathematical distance versus an Approximate Nearest Neighbor index seek using VECTOR_SEARCH). It does absolutely nothing to refresh or regenerate the stale underlying embedding payload column.

C (Hourly T-SQL Job for all rows): Regenerating text embeddings from scratch for all two million articles every single hour is highly inefficient and creates an immense, continuous processing bottleneck on SalesDB CPU resources.

D (Create an AI_GENERATE_EMBEDDINGS Trigger): While database triggers capture changes instantaneously, embedding generation relies on an external network API call to an AI model. Forcing synchronous HTTP calls inside a database DML trigger for frequent updates halts the transactional loop, inflates connection lock times, and drastically spikes CPU utilization.

Question: 23 Exam Author

You have a GitHub Codespaces environment that has GitHub Copilot Chat installed and is connected to a SQL database in Microsoft Fabric named DB1. DB1 contains tables named Sales.Orders and Sales.Customers.
You use GitHub Copilot Chat in the context of DB1.
A company policy prohibits sharing customer Personally Identifiable Information (PII), secrets, and query result sets with any AI service.
You need to use GitHub Copilot Chat to write and review Transact-SQL code for a new stored procedure that will join Sales.Orders to Sales.Customers and return customer names and email addresses. The solution must NOT share the actual data in the tables with GitHub Copilot Chat.
What should you do?

A. From Sales.Customers, paste several rows that include email addresses into a chat, so that GitHub Copilot Chat can infer edge cases.

B. Run a SELECT statement that returns customer names and email addresses and provide the result set to GitHub Copilot Chat so that GitHub Copilot Chat can generate the stored procedure.

C. Provide the database connection string to GitHub Copilot Chat so that GitHub Copilot Chat can validate the stored procedure.

D. Ask GitHub Copilot Chat to generate the stored procedure by using schema details only.

Answer: D

Explanation:

D. Ask GitHub Copilot Chat to generate the stored procedure by using schema details only.

To adhere strictly to company security policies prohibiting the sharing of PII, secrets, and data results with an AI model, you only need to provide the metadata or structural blueprint (schema details) of the relevant tables. GitHub Copilot Chat can fully understand table structures, column names (such as CustomerID, CustomerName, and EmailAddress), data types, and primary/foreign key relationships. Providing just this schema definition gives the AI agent all the context required to craft an accurate, optimized inner-join stored procedure without ever exposing actual backend transactional records.

Why the other options are incorrect:

A (Paste rows that include email addresses): Copying and pasting actual customer email addresses directly into the chat pane explicitly transmits live, real-world customer PII to an external AI training/processing endpoint, causing a major policy violation.

B (Provide the query result set): The prompt states that the company policy explicitly bans sharing query result sets with any AI service. Providing an interactive result block with structural data compromises data compliance.

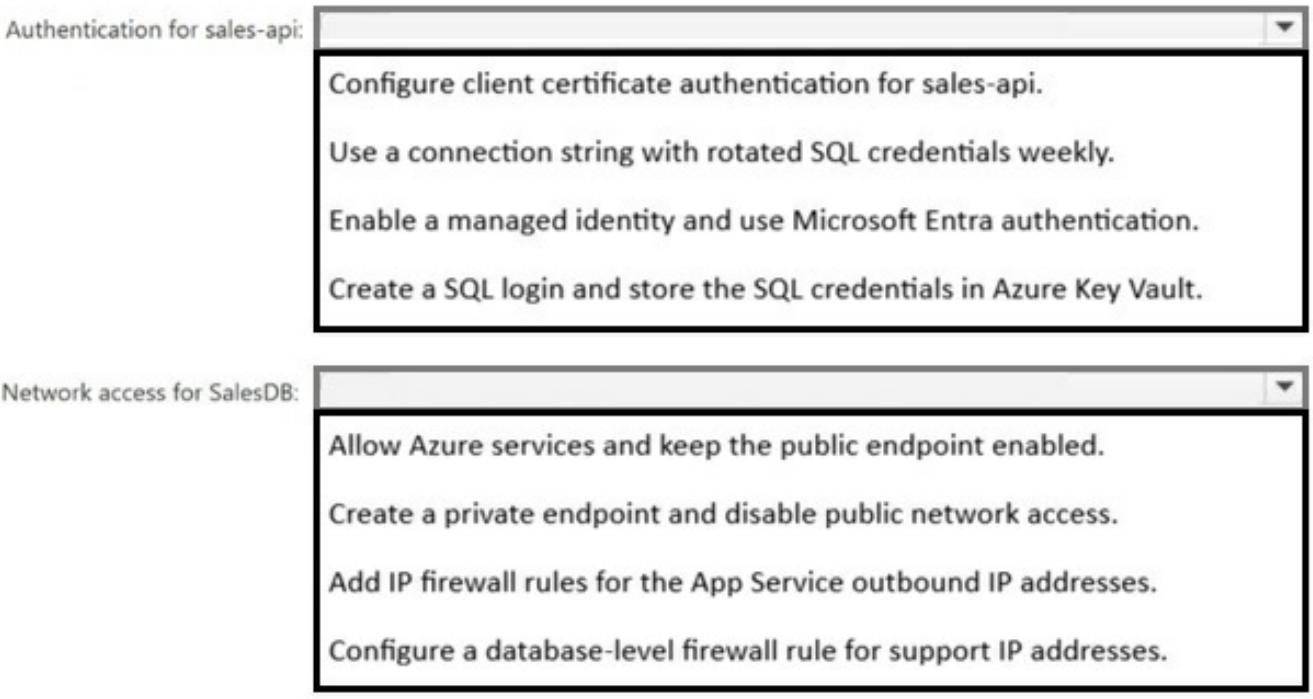
C (Provide the database connection string): Connection strings typically embed highly confidential database secrets (like passwords, access keys, or internal routing endpoints). Exposing these strings breaks the rule against sharing secrets with AI tools.

Question: 24 Exam Author

HOTSPOT -
You have an Azure subscription. The subscription contains an Azure SQL database named SalesDB and an Azure App Service app named sales-api. sales-api uses virtual network integration to a subnet named vnet-prod/subnet-app and reads from SalesDB.

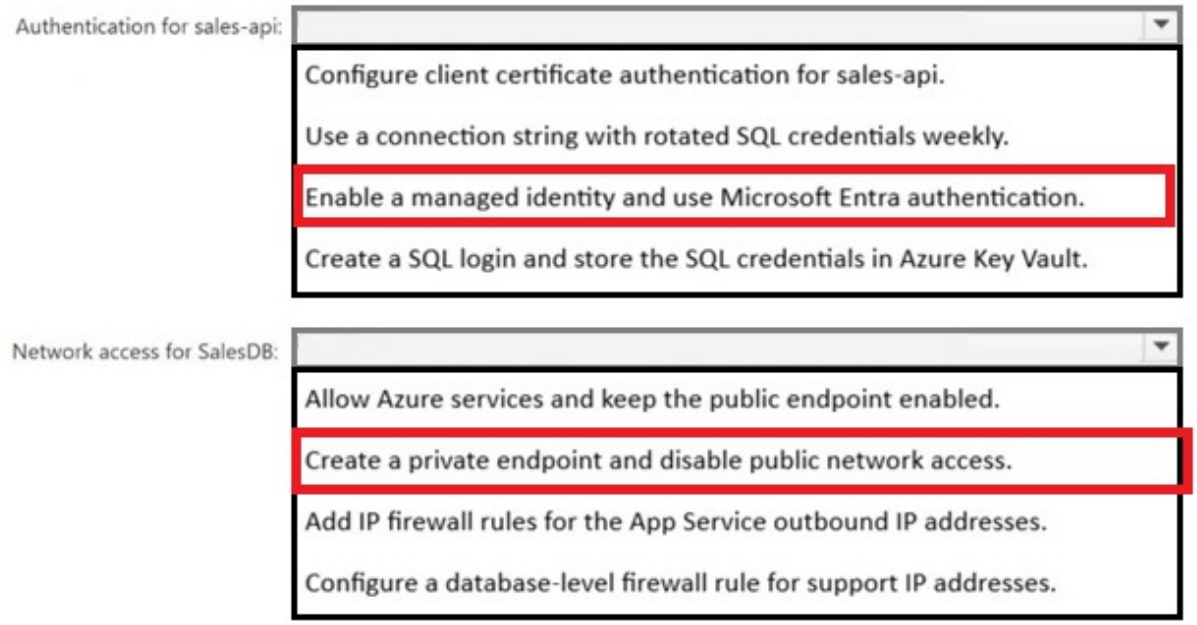
You need to configure authentication and network access to meet the following requirements:
Ensure that sales-api connects to SalesDB by using passwordless authentication.
Ensure that all the database traffic remains within the subscription.
The solution must minimize administrative effort.
What should you configure? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area



Answer:

Answer Area



Explanation:

Authentication for sales-api: **Enable a managed identity and use Microsoft Entra authentication.**

Enabling a managed identity eliminates the operational overhead of hardcoding, rotating, or securing plaintext passwords or secrets. It configures a passwordless, token-based Microsoft Entra ID authentication

pipeline that automatically establishes a secure cryptographic relationship between the App Service (sales-api) and SalesDB.

Network access for SalesDB: **Create a private endpoint and disable public network access.**

Implementing a private endpoint maps a private IP address from your Virtual Network (VNet) directly to the SalesDB instance. Disabling public network access locks down the database entirely, guaranteeing that traffic remains strictly over Microsoft's internal private backbone fabric and completely blocking all inbound brute-force attempts or scanning from the public internet.

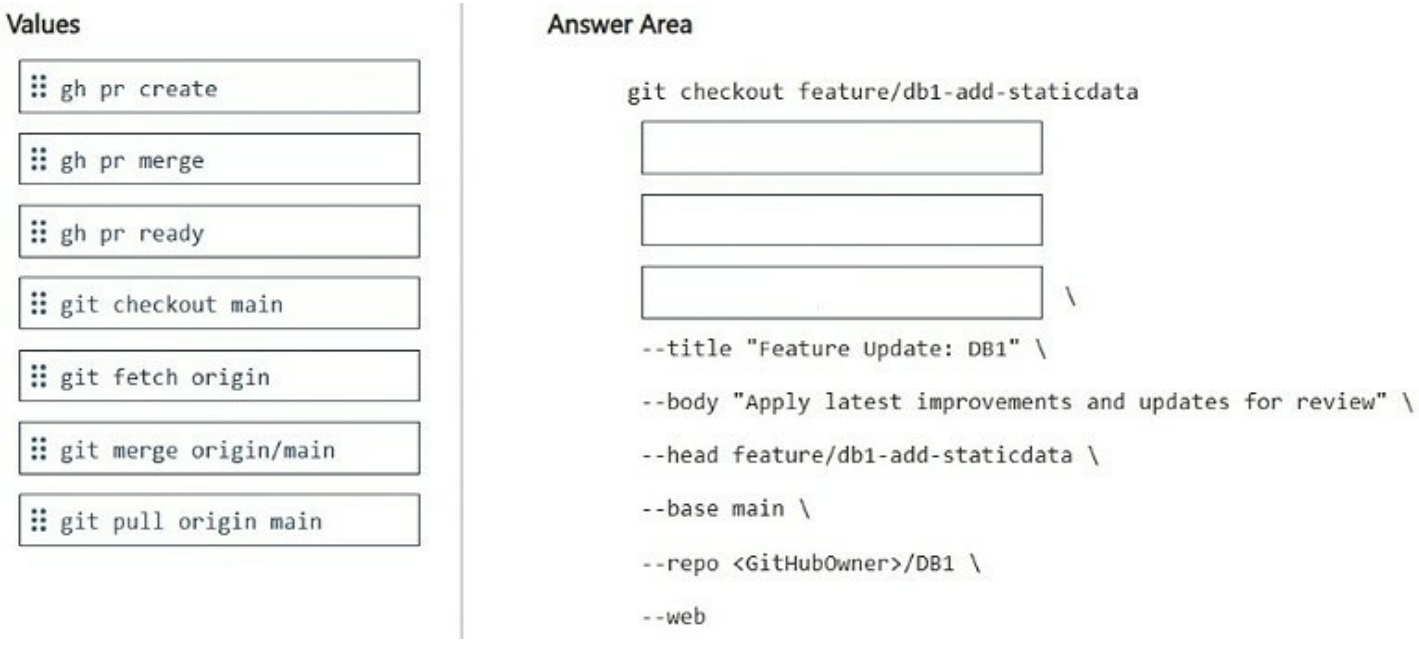
Why the others are incorrect:

Relying on standard SQL logins — even if protected inside Azure Key Vault or rotated weekly — still creates administrative management burdens and maintains a vulnerability footprint for credential leaks. Client certificate authentication does not natively map to managed Azure database users.

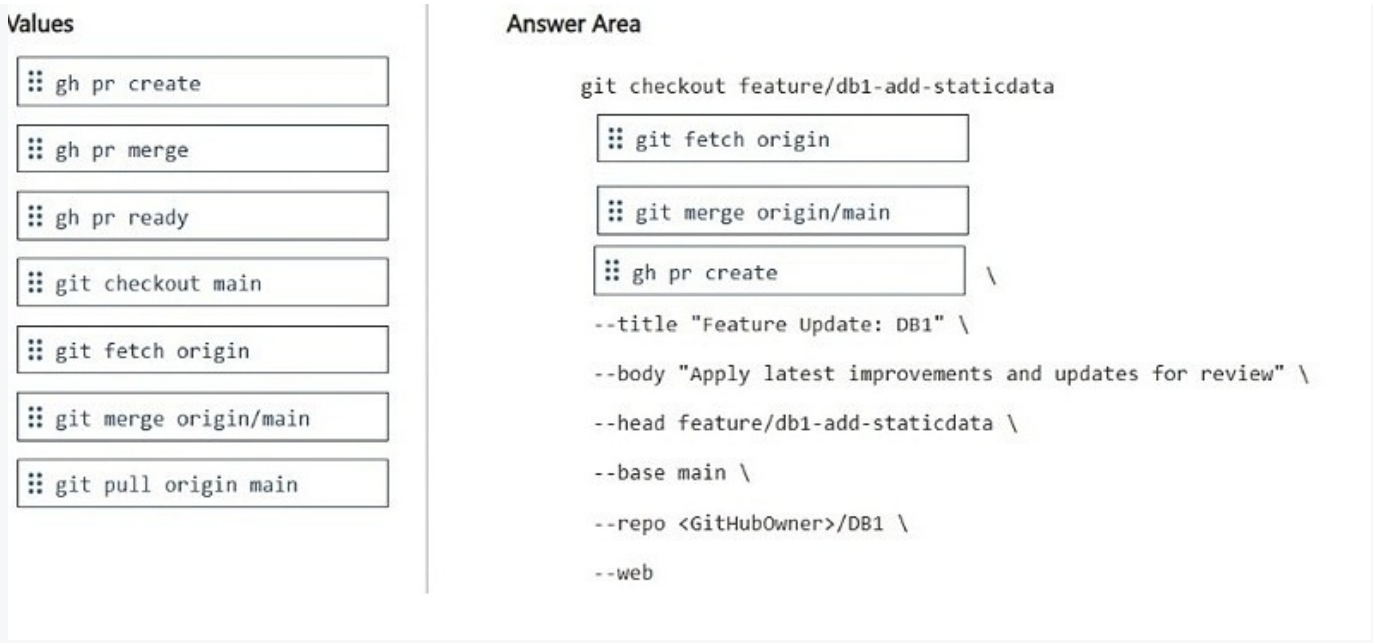
Keeping the public endpoint enabled or relying on App Service outbound IP firewall rules leaves a route open to the public internet space. This fails to provide maximum structural perimeter isolation compared to a dedicated, localized Private Link solution.

Question: 25 **Exam Author**

DRAG DROP -
You have a database named DB1. The schema is stored in a GitHub repository as an SDK-style SQL database project.
You use a feature branch workflow to deploy changes to DB1.
You need to update the local feature branch with the latest changes to main, and then create a pull request to merge the feature branch into main for review.
How should you complete the GitHub CLI script? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Box 1 (**git fetch origin**): Retrieves the latest references, history, and branch trackers directly from the remote GitHub repository without overwriting any active local file modifications.

Box 2 **(git merge origin/main**): Merges the freshly retrieved history of the remote main branch tracking stream (origin/main) cleanly into the current checked-out local working feature branch (feature/db1-add-staticdata). This step ensures that any upstream merge conflicts are resolved locally prior to creating a pull request.

Box 3 **(gh pr create):** The definitive GitHub CLI command syntax used to spin up a pull request. It explicitly references all trailing line-continuation backslash variables (--title, --body, --head, --base, --repo, and --web).

Question: 26 **Exam Author**

You have an Azure SQL database that contains tables named dbo.ProductDocs and dbo.ProductDocsEmbeddings.
dbo.ProductDocs contains product documentation and the following columns:
DocID (int)
Title (nvarchar(200))
Body (nvarchar(max))
LastModified (datetime2)
The documentation is edited throughout the day.
dbo.ProductDocsEmbeddings contains the following columns:
DocID (int)
ChunkOrder (int)
ChunkText (nvarchar(max))
Embedding (vector(1536))
The current embedding pipeline runs once per night.
You need to ensure that embeddings are updated every time the underlying documentation content changes. The solution must NOT require a nightly batch process.
What should you include in the solution?

A. fixed-size chunking
B. a smaller embedding model
C. table triggers
D. change tracking on dbo.ProductDocs

Answer: D

Explanation:

D. change tracking on dbo.ProductDocs.

Enabling Change Tracking on the dbo.ProductDocs table allows an external processing application or service to incrementally query only the rows that have been inserted, updated, or deleted since the last synchronization token. This avoids the need for heavy nightly batch jobs or full table scans, enabling near real-time, lightweight vector embedding maintenance.

Why the other answers are incorrect:

A (Fixed-size chunking): Chunking defines how the textual data is split up structurally into blocks before it is processed by an embedding model. It does not provide any background system mechanism to detect data alterations.

B (A smaller embedding model): While an alternative model might reduce operational cost or generation latency, it doesn't resolve the core architecture requirement of identifying changed rows in real-time.

C (Table triggers): Although database triggers run immediately on row mutations, initiating network API calls to external embedding endpoints inside a synchronous data manipulation (DML) trigger blocks transactions, spikes CPU utilization, and reduces scalability.

Question: 27 **Exam Author**

HOTSPOT -
You have a database named db1. The schema is stored in a Git repository as an SDK-style SQL database project. The repository contains the following GitHub Action workflow.

```yaml
name: Database CI/CD
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout code
      uses: actions/checkout@v3
    - name: Setup .NET
      uses: actions/setup-dotnet@v3
      with:
        dotnet-version: '8.x'
    - name: Build
      run: dotnet build db1.sqlproj --configuration Release
    - name: Deploy
      run: |
        SqlPackage /Action:Publish \
          /SourceFile:./bin/Release/db1.dacpac \
          /TargetConnectionString:"${{ secrets.Target_Connection_String }}"
  unit-tests:
    runs-on: ubuntu-latest
    needs: build-and-deploy
    if: github.ref == 'refs/heads/main'
    steps:
    - name: Checkout code
      uses: actions/checkout@v3
    - name: Setup .NET
      uses: actions/setup-dotnet@v3
      with:
        dotnet-version: '8.x'
    - name: Run unit tests
      run: dotnet test UnitTests.csproj --configuration Release
```

For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
Unit tests run automatically whenever changes are pushed to `main`.	○	○
Schema validation occurs during the `Build` step.	○	○
Schema validation occurs during the `Deploy` step.	○	○

Answer:

Answer Area

Statements	Yes	No
Unit tests run automatically whenever changes are pushed to `main`.	○	○
Schema validation occurs during the `Build` step.	○	○
Schema validation occurs during the `Deploy` step.	○	○

Explanation:

Unit tests run automatically whenever changes are pushed to main

Yes

Why it is correct: In automated GitHub Actions or Azure DevOps pipelines, workflows configured with push triggers targeting specific branches (such as push.branches: [main]) execute all non-conditional testing blocks directly inside the job sequence as soon as new code is committed upstream.

Schema validation occurs during the Build step

Yes

When compiling an SDK-style SQL database project (using commands like dotnet build db1.sqlproj), the compilation compiler natively constructs and analyzes the database structural model. It validates object dependencies, column references, data types, and constraint syntax. Therefore, core static schema-level validation happens entirely during the build step before any artifact is created.

Schema validation occurs during the Deploy step

No

The deployment step typically utilizes a deployment engine utility (like SqlPackage /Action:Publish) to take an already validated .dacpac artifact and apply it incrementally to a live target database instance. This is a synchronization and script generation execution step — it does not serve as the baseline verification phase for your project's code logic.

Question: 28 **Exam Author**

You have a Microsoft SQL Server 2025 instance that contains a database named SalesDB. SalesDB supports a Retrieval Augmented Generation (RAG) pattern for internal support tickets. The SQL Server instance runs without any outbound network connectivity.
You plan to generate embeddings inside the SQL Server instance and store them in a table for vector similarity queries.
You need to ensure that only a database user account named AIApplicationUser can run embedding generation by using the model.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Grant the CONTROL permission on SalesDB to AIApplicationUser.
B. Create a database audit specification on SalesDB owned by AIApplicationUser.
C. Grant the EXECUTE permission on the external model project to AIApplicationUser.
D. Create an external model project by using ONNX runtime and local paths.
E. Create an external model project that points to a Microsoft Foundry REST endpoint.

Answer: CD

Explanation:

C. Grant the EXECUTE permission on the external model project to AIApplicationUser.

D. Create an external model project by using ONNX runtime and local paths.

Why C is correct (Access Control & Security): To adhere to the security principal of least privilege, you should not grant wide structural administrative permissions (like database CONTROL or db_owner). In SQL Server 2025, external model configurations behave like secured objects; explicitly executing GRANT EXECUTE ON EXTERNAL MODEL::[ModelName] TO [AIApplicationUser] ensures that only that specific user account can invoke the AI_GENERATE_EMBEDDINGS function against that specific model.

Why D is correct (Infrastructure & Connectivity): Because the SQL Server 2025 instance runs without any outbound network connectivity, it cannot communicate with external remote cloud REST endpoints like Azure OpenAI or Azure AI Foundry. The solution requires a native, local execution method. SQL Server 2025 introduces local ONNX Runtime deployment integration, which evaluates localized text-embedding models entirely on the host machine using localized disk paths.

Why the other choices are incorrect:

A (CONTROL permission): This provides sweeping administrative powers over the entire SalesDB database, completely violating the least-privilege security constraint.

B (Database Audit Specification): Database audit configurations track, log, and record activity for compliance evaluation. They cannot act as an inline query gatekeeper or enforce access blockades to prevent users from executing a model.

E (Microsoft Foundry REST endpoint): REST endpoints explicitly rely on operational HTTP/HTTPS outbound traffic networks, which completely fails because the hosting environment is network-isolated.

Question: 29 **Exam Author**

DRAG DROP -
You have an Azure SQL database that supports an OLTP application.
You need to write Transact-SQL code that returns blocking chain details. The output must return only sessions that are blocked or are blocking other sessions.
How should you complete the code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Answer:

Explanation:

Box 1 **(FROM sys.dm_exec_requests)**: Placed within the localized XML subquery expression to loop through running instances and concatenate the IDs of all sessions that are currently waiting on a resource lock from a blocker.

Box 2 **(LEFT OUTER JOIN sys.dm_exec_requests)**: Vital for detecting "sleeping" transactions. If an application opens a transaction block and becomes inactive without calling COMMIT or ROLLBACK, it won't have an active row inside sys.dm_exec_requests. A LEFT OUTER JOIN preserves these idle sessions in the final list.

Box 3 **(OUTER APPLY sys.dm_exec_sql_text (r.sql_handle)**): Resolves the internal batch statement string associated with active queries. Utilizing OUTER APPLY ensures that if a session has no current active request handle, the row is still returned rather than getting filtered out.

Box 4 (**OUTER APPLY sys.dm_exec_input_buffer(s.session_id, NULL)):** Replaces old legacy DBCC INPUTBUFFER commands. Passing NULL

as the second parameter retrieves the very last batch payload sent down the client socket connection channel for that session, pinpointing the root query causing the database deadlock or lock escalation.

Question: 30 **Exam Author**

You have an Azure SQL database that supports a customer-facing API. The API calls a stored procedure named dbo.GetCustomerOrders thousands of times per hour.
After a deployment that updated indexes and statistics, users report that the API endpoint backed by dbo.GetCustomerOrders is slower. In Query Store, the same query now has two persisted execution plans. During the last hour, the newer plan had a significantly higher average duration and CPU time than the older plan.
You need to restore the previous performance quickly, without changing the API code.
Which Transact-SQL command should you run?

A. EXEC sys.sp_query_store_set_hints

B. DBCC FREEPROCCACHE

C. EXEC sp_query_store_force_plan

D. ALTER DATABASE

Answer: C

Explanation:

C. EXEC sp_query_store_force_plan.

The scenario describes a classic case of Plan Regression (a parameter sniffing or suboptimal compilation issue where a worse plan replaces a highly efficient one). Since Query Store has already tracked and persisted both plans, running sp_query_store_force_plan allows you to immediately force the SQL Server optimizer to use the older, high-performing execution plan. This completely restores original performance instantly without requiring any code deployment or changes to the API client application.

Why the other options are incorrect:

A (sys.sp_query_store_set_hints): Query Store hints can shape how a plan is compiled (such as appending a specific MAXDOP or RECOMPILE rule). However, crafting and testing a hint takes more time, and it does not directly re-apply a specific, verified, historically known plan like forcing does.

B (DBCC FREEPROCCACHE): Clearing the plan cache evicts all cached execution plans. While this forces dbo.GetCustomerOrders to recompile, it is highly likely that the optimizer will simply recreate the same bad, heavy-duration plan based on current statistics. Additionally, it causes a global performance dip across the entire database as every query is forced to recompile simultaneously.

D (ALTER DATABASE): While database-scoped configurations (like automatic tuning features) can be altered here, running a raw ALTER DATABASE statement requires further arguments and does not target the exact plan regression anomaly inside the dbo.GetCustomerOrders execution history in a quick, surgical fashion.

Question: 31 **Exam Author**

HOTSPOT -
You have an Azure SQL database that has Query Store enabled.
Query Performance Insight shows that one stored procedure has the longest runtime. The procedure runs the following parameterized query.

```
CREATE OR ALTER PROCEDURE dbo.GetRecentOrders
  @CustomerId int,
  @SinceDate  datetime2
AS
SELECT TOP (50)
    o.OrderId, o.OrderDate, o.Status, o.TotalAmount
FROM dbo.Orders AS o
WHERE o.CustomerId = @CustomerId
  AND o.OrderDate >= @SinceDate
ORDER BY o.OrderDate DESC;
```

The dbo.Orders table has approximately 120 million rows. CustomerId is highly selective, and OrderDate is used for

range filtering and sorting.
You have the following indexes:
Clustered index: PK_Orders on (OrderId)
Nonclustered index: IX_Orders_OrderDate on (OrderDate) with no included columns
An actual execution plan captured from Query Store for slow runs shows the following:
An index seek on IX_Orders_OrderDate followed by a Key Lookup (Clustered) on PK_Orders for CustomerId, Status, and TotalAmount
A sort operator before Top (50), because the results are ordered by OrderDate DESC
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
To avoid the explicit sort for the query, create a nonclustered index on (CustomerId, OrderDate DESC) that includes (Status, TotalAmount).	○	○
To eliminate the sort and make the query use an ordered seek, add CustomerId as an included column to IX_Orders_OrderDate.	○	○
The plan indicates a bottleneck from a suboptimal query plan, rather locking or blocking.	○	○

Answer:

Answer Area

Statements	Yes	No
To avoid the explicit sort for the query, create a nonclustered index on (CustomerId, OrderDate DESC) that includes (Status, TotalAmount).	○	○
To eliminate the sort and make the query use an ordered seek, add CustomerId as an included column to IX_Orders_OrderDate.	○	○
The plan indicates a bottleneck from a suboptimal query plan, rather locking or blocking.	○	○

Explanation:

Avoid explicit sort with (CustomerId, OrderDate DESC)

Yes

When a query filters by an equality condition (e.g., WHERE CustomerId = @Id) and sorts the output (e.g., ORDER BY OrderDate DESC), creating a composite index with those key columns in that exact order allows the SQL Server query optimizer to perform an Index Seek. Because the index data is already physically ordered by OrderDate within each CustomerId, the database engine fetches the data pre-sorted, completely bypassing a costly, explicit Sort operator in the execution plan.

Adding CustomerId as an included column to eliminate sort

No

Included columns (INCLUDE) only live at the leaf level of an index b-tree structure; they are not part of the sorted index key columnsCustomerId is just an included column, the index cannot be used to seek directly to that customer's records, nor will the data be organized in a way that eliminates the sorting step. To allow an ordered seek, CustomerId must be a key column, not an included column.

Bottleneck from a suboptimal query plan rather than locking.

Yes

An execution plan shows the structural strategy (operators like table scans, hashes, or explicit sort operations) chosen by the optimizer to run a query. High costs within these operators indicate a suboptimal query plan design (such as a missing index causing a tempdb sort spill). Conversely, runtime concurrency issues like transactional locking or thread blocking are environmental wait states that are not explicitly modeled or captured by a standard graphical query execution plan.

Exam : **DP-800**

Title : Developing AI-Enabled Database Solutions

Version : DEMO

1. Topic 1, Contoso Case Study

Existing Environment

Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database.

The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleHealthSummary	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the unmask permission.

Requirements
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following;
• Al workloads
• Vector search
• Modernized API access
• Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.
SELECT VehicleTd, Lastupdatedutc, EngineStatus, BatteryHealth FROM dbo.VehicleHealthSumary where fleetld - gFleetld ORDER BV LastUpdatedUtc DESC;
You review the execution plan and discover that the plan shows a clustered index scan.
vehicleincident Reports often contains details about the weather, traffic conditions, and location.
Analysts report that it is difficult to find similar incidents based on these details.

Planned Changes
Contoso wants to modernize Fleet Intelligence Platform to support Al-powered semantic search over incident reports.

Security Requirements
Contoso identifies the following telemetry requirements:
• Telemetry data must be stored in a partitioned table.
• Telemetry data must provide predictable performance for ingestion and retention operations.
• latitude, longitude, and accuracy JSON properties must be filtered by using an index seek. Contoso identifies the following maintenance data requirements:
• Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModif reduce column to the time of the change.
• Avoid recursive updates.

Al Search, Embedding's, and Vector indexing
The development learn at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.
Contoso identifies the following requirements for querying data in the FeedbackJson column of the customer-Feedback table:
• Extract the customer feedback text from the JSON document.
• Filter rows where the JSON text contains a keyword.
• Calculate a fuzzy similarity score between the feedback text and a known issue description.
• Order the results by similarity score, with the highest score first.

You need to generate embeddings to resolve the issues identified by the analysts.
Which column should you use?
A. vehicleLocation
B. incidentDescrlption

C. incidentType
D. SeverityScore
Answer: B
Explanation:
The correct column to use for generating embeddings is incidentDescrlption because embeddings are intended to represent the semantic meaning of rich textual content, not simple categorical, numeric, or location-only values. Microsoft's DP-800 study guide explicitly includes skills such as identifying which columns to include in embeddings, generating embeddings, and implementing semantic vector search for scenarios where users need to find similar records based on meaning rather than exact matches.
In this scenario, analysts report that it is difficult to find similar incidents based on details such as weather, traffic conditions, and location. Those are descriptive context elements that are typically captured in a free-text incident description field. An embedding generated from incidentDescrlption can encode the semantic relationships among these narrative details, making it suitable for similarity search, semantic search, and RAG retrieval. Microsoft documentation on vectors and embeddings explains that embeddings are generated from text data and then stored for vector search to find semantically related items.
The other options are weaker choices:
vehicleLocation is too narrow and usually better handled with geospatial filtering, not embeddings.
incidentType is likely categorical and too low in semantic richness.
SeverityScore is numeric and not appropriate as the primary source for semantic embeddings.
Microsoft also notes that when multiple useful attributes exist, you can either embed each text column separately or concatenate relevant text fields into one textual representation before generating the embedding. But among the options given, the best and most exam-aligned answer is the textual narrative column: incidentDescrlption.

2.You need to recommend a solution for the development team to retrieve the live metadata. The solution must meet the development requirements.
What should you include in the recommendation?
A. Export the database schema as a .dacpac file and load the schema into a GitHub Copilot context window.
B. Add the schema to a GitHub Copilot instruction file.
C. Use an MCP server
D. Include the database project in the code repository.
Answer: C
Explanation:
The best recommendation is to use an MCP server. In the official DP-800 study guide, Microsoft explicitly lists skills such as configuring Model Context Protocol (MCP) tool options in a GitHub Copilot session and connecting to MCP server endpoints, including Microsoft SQL Server and Fabric Lakehouse. That makes MCP the exam-aligned mechanism for enabling AI-assisted tools to work with live database context rather than static snapshots.
This also matches the stated development requirement: the team will use Visual Studio Code and GitHub Copilot and needs to retrieve live metadata from the databases. Microsoft's documentation for GitHub Copilot with the MSSQL extension explains that Copilot works with an active database connection, provides schema-aware suggestions, supports chatting with a connected database, and

adapts responses based on the current database context. Microsoft also documents MCP as the standard way for AI tools to connect to external systems and data sources through discoverable tools and endpoints.
The other options do not satisfy the “live metadata” requirement as well:
A .dacpac is a point-in-time schema artifact, not live metadata.
A Copilot instruction file provides guidance, not live database discovery.
Including the database project in the repository helps source control and deployment, but it still does not provide live database metadata by itself.

3.You need to recommend a solution that will resolve the ingestion pipeline failure issues.
Which two actions should you recommend? Each correct answer presents part of the solution. NOTE: Each correct selection is worth one point.
A. Enable snapshot isolation on the database.
B. Use a trigger to automatically rewrite malformed JSON.
C. Add foreign key constraints on the table.
D. Create a unique index on a hash of the payload.
E. Add a check constraint that validates the JSON structure.
Answer: D, E
Explanation:
The two correct actions are D and E because the ingestion failures are caused by malformed JSON and duplicate payloads, and these two controls address those two problems directly. Microsoft’s JSON documentation states that SQL Server and Azure SQL support validating JSON with ISJSON, and Microsoft specifically recommends using a CHECK constraint to ensure JSON text stored in a column is properly formatted.
For the duplicate-payload issue, creating a unique index on a hash of the payload is the appropriate design. Microsoft documents using hashing functions such as HASHBYTES to hash column values, and SQL Server allows a deterministic computed column to be used as a key column in a UNIQUE constraint or unique index. That makes a persisted hash-based computed column plus a unique index a practical and exam-consistent way to reject duplicate payloads efficiently.
The other options do not solve the stated root causes:
Snapshot isolation addresses concurrency behavior, not malformed JSON or duplicate payload detection.
A trigger to rewrite malformed JSON is not the right integrity control and is brittle.
Foreign key constraints enforce referential integrity, not JSON validity or duplicate-payload prevention

4.HOTSPOT
You need to meet the development requirements for the FeedbackJson column
How should you complete the Transact SQL query? To answer, select the appropriate options in the answer area. NOTE: Each correct selection is worth one point.

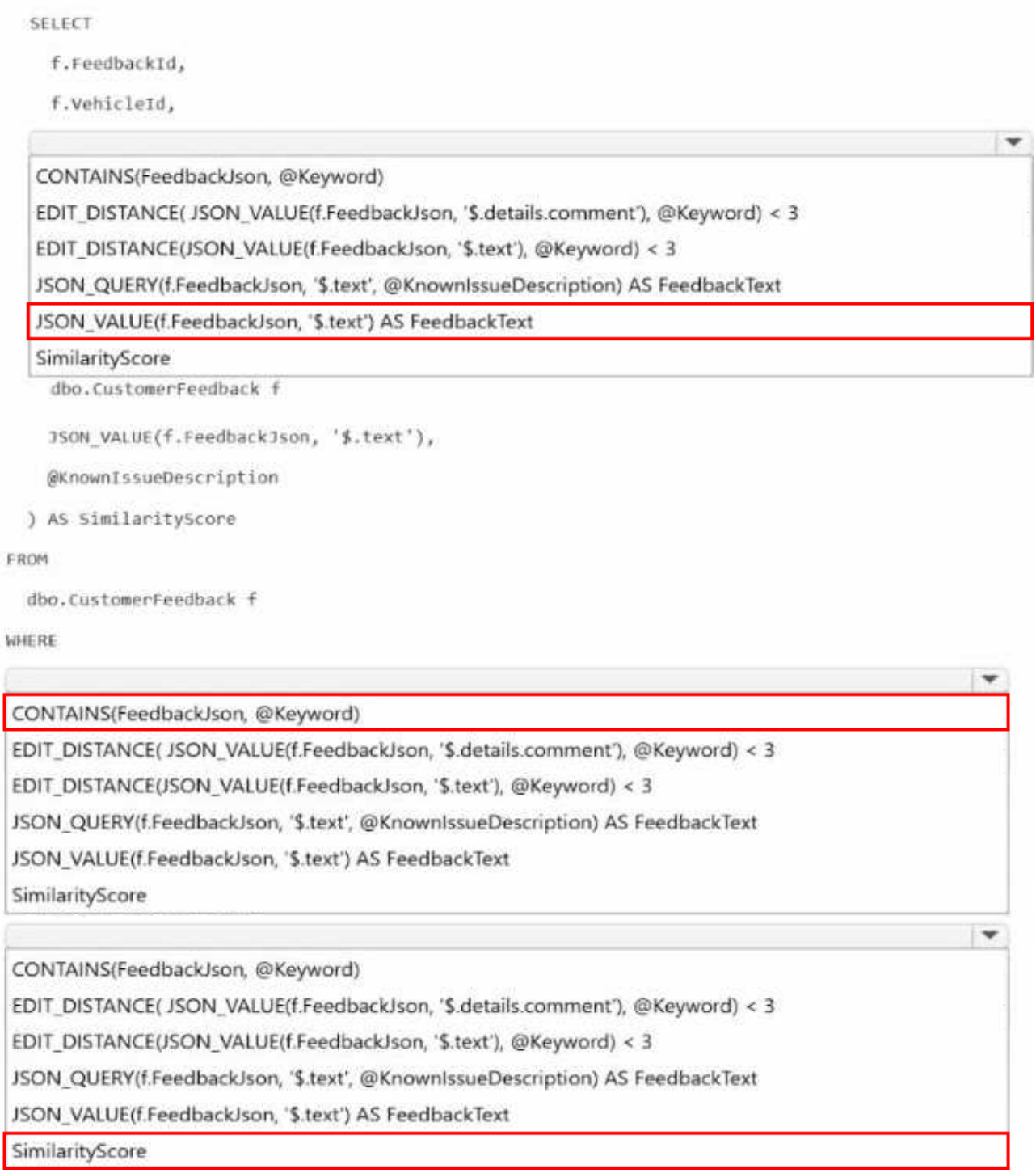
Answer Area

```
SELECT
  f.FeedbackId,
  f.VehicleId,
  [                                                                    ]
  dbo.CustomerFeedback f
  JSON_VALUE(f.FeedbackJson, '$.text'),
  @KnownIssueDescription
) AS SimilarityScore
FROM
  dbo.CustomerFeedback f
WHERE
  [                                                                    ]
  [                                                                    ]
```

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

Answer:

Answer Area



Explanation:

JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText

CONTAINS(FeedbackJson, @Keyword)

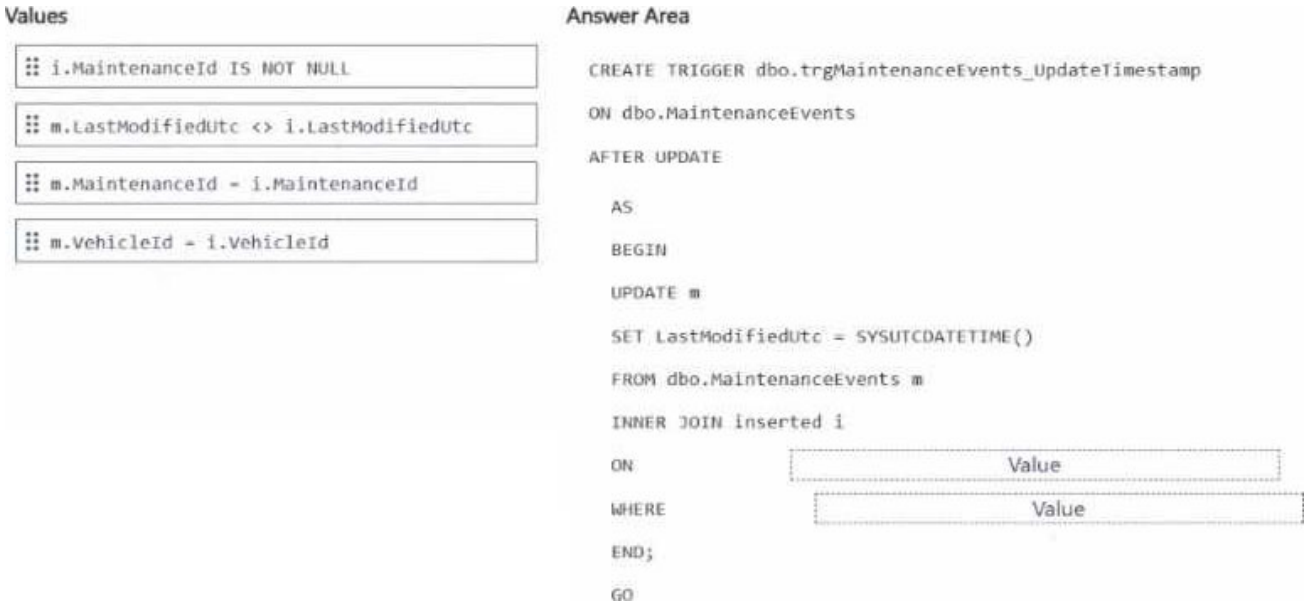
SimilarityScore

These three selections are the correct way to complete the query because they align exactly with the stated requirements for the FeedbackJson column.

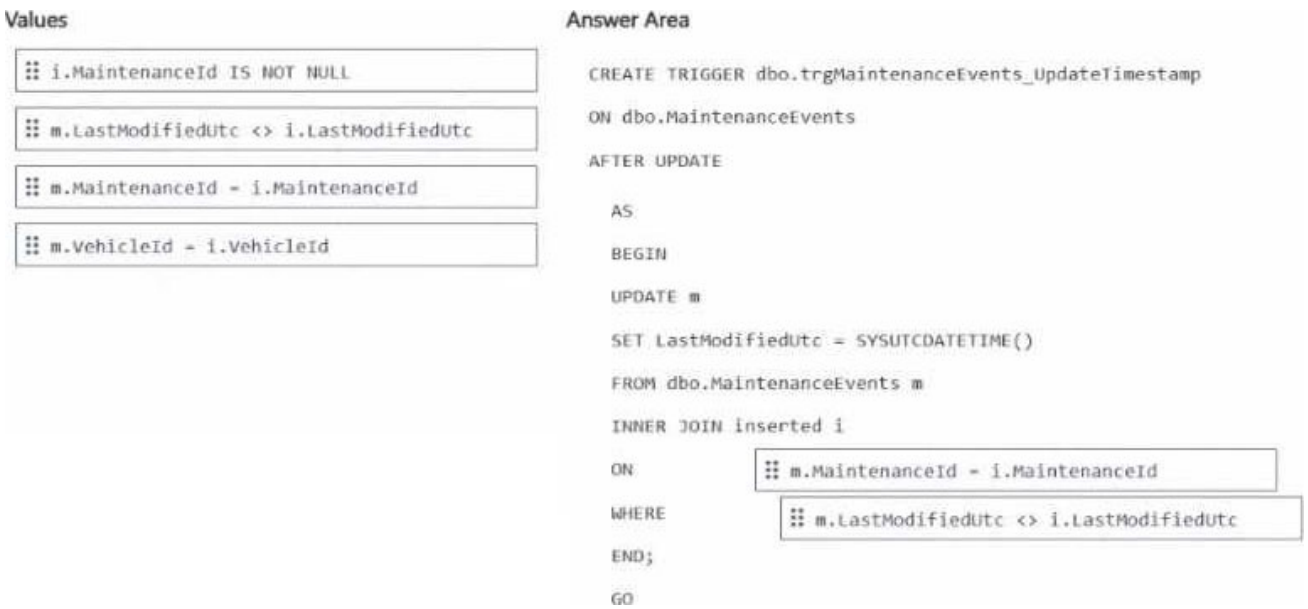
First, to extract the customer feedback text from the JSON document, the correct expression is JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText. Microsoft documents that JSON_VALUE is

used to extract a scalar value from JSON, while JSON_QUERY is used for returning an object or array. Since $.text is the textual feedback string, JSON_VALUE is the correct function.
Second, to filter rows where the JSON text contains a keyword, the best choice is CONTAINS(FeedbackJson, @Keyword). The scenario explicitly states that FeedbackJson already has a full-text index, and Microsoft documents that CONTAINS is the full-text predicate used in the WHERE clause to search full-text indexed character data. That makes it more appropriate than using EDIT_DISTANCE for keyword filtering.
Third, to order the results by similarity score, highest first, the correct item is SimilarityScore in the ORDER BY clause, which would be paired with DESC in the query. This matches the requirement to sort by the computed fuzzy similarity value. The DP-800 study guide specifically includes writing queries that use fuzzy string matching functions such as EDIT_DISTANCE, which supports the earlier computed SimilarityScore expression in the query.

5.DRAG DROP
You need to meet the database performance requirements for maintenance data
How should you complete the Transact-SQL code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content. NOTE: Each correct selection is worth one point.



Answer:



Explanation:

ON → m.maintenanceId = i.maintenanceId

WHERE → m.LastModifiedUtc <> i.LastModifiedUtc

The correct drag-and-drop completion is:

ON m.maintenanceId = i.maintenanceId

WHERE m.LastModifiedUtc <> i.LastModifiedUtc

This satisfies the requirement to ensure that when a row in MaintenanceEvents changes, the corresponding LastModifiedUtc value is updated to the current system time, while also helping avoid unnecessary repeat updates.

The inserted pseudo-table in a SQL Server AFTER UPDATE trigger contains the rows that were just updated. To update the matching row in the base table correctly, the trigger must join the target table row to the corresponding row in inserted by the table's primary key. In this schema, MaintenanceId is the primary key for MaintenanceEvents, so the correct join is m.maintenanceId = i.maintenanceId. Joining on VehicleId would be incorrect because multiple maintenance rows could exist for the same vehicle, which could update unintended rows. Microsoft's trigger documentation explains that inserted and deleted are used to work with the affected rows and that multi-row logic should be based on proper key matching.

The WHERE m.LastModifiedUtc <> i.LastModifiedUtc predicate is used to prevent the trigger from re-updating rows where the timestamp already matches the value in inserted. That reduces redundant writes and supports the requirement to avoid recursive or repeated update behavior. In practice, this means the trigger updates only rows whose current stored timestamp differs from the just-updated version. This is the exam-appropriate pattern for a self-updating timestamp column in an AFTER UPDATE trigger.